



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)
دانشکده مهندسی کامپیوتر

پایان نامه کارشناسی ارشد
گرایش هوش مصنوعی و رباتیکز

بهبود سرعت مدل های پخشی

نگارش

عطیه غفارلوی مقدم

استاد راهنما

دکتر احمد نیک آبادی

دی ماه ۱۴۰۳

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

به نام خدا

تعهدنامه اصالت اثر

تاریخ: دی ماه ۱۴۰۳

اینجانب عطیه غفارلوی مقدم متعهد می‌شوم که مطالب مندرج در این پایان‌نامه حاصل کار پژوهشی اینجانب تحت نظارت و راهنمایی اساتید دانشگاه صنعتی امیرکبیر بوده و به دستاوردهای دیگران که در این پژوهش از آنها استفاده شده است مطابق مقررات و روال متعارف ارجاع و در فهرست منابع و مآخذ ذکر گردیده است. این پایان‌نامه قبلاً برای احراز هیچ مدرک هم‌سطح یا بالاتر ارائه نگردیده است. در صورت اثبات تخلف در هر زمان، مدرک تحصیلی صادر شده توسط دانشگاه از درجه اعتبار ساقط بوده و دانشگاه حق پیگیری قانونی خواهد داشت.

کلیه نتایج و حقوق حاصل از این پایان‌نامه متعلق به دانشگاه صنعتی امیرکبیر می‌باشد. هرگونه استفاده از نتایج علمی و عملی، واگذاری اطلاعات به دیگران یا چاپ و تکثیر، نسخه‌برداری، ترجمه و اقتباس از این پایان‌نامه بدون موافقت کتبی دانشگاه صنعتی امیرکبیر ممنوع است. نقل مطالب با ذکر مآخذ بلامانع است.

عطیه غفارلوی مقدم

امضا

چکیده

مدل‌های پخشی به عنوان یک خانواده‌ی قدرتمند از مدل‌های مولد با نتایج قابل توجه در حوزه‌های مختلف تصویر، صوت، متن و گراف، در سال‌های اخیر مورد توجه بسیاری از محققین یادگیری ماشین قرار گرفته‌اند. همه‌ی مدل‌های مطرح شده در این خانواده از یک رویکرد مشابه پیروی می‌کنند و آن هم تخریب داده با نویز در یک فرآیند گام به گام و سپس بازگرداندن این فرآیند برای تولید نمونه‌های جدید از داده می‌باشد. در این گزارش ابتدا به بررسی دقیق سه دسته‌ی مطرح مدل‌های پخشی و شباهت‌ها و تفاوت‌های آنها می‌پردازیم. پس از معرفی این مدل‌ها، به بررسی رابطه‌ی میان آنها و انواعی دیگر از مدل‌های مولد می‌پردازیم و در نهایت به تعدادی از چالش‌های مطرح در خصوص این مدل‌ها اشاره می‌کنیم.

واژه‌های کلیدی:

مدل‌های مولد، مدل‌های پخشی، مدل‌های مولد مبتنی بر امتیاز، مدل‌های پخشی نویززدای احتمالاتی

فهرست مطالب

صفحه

عنوان

۱	مقدمه	۱
۲	مدل‌های پخشی و رویکردهای افزایش سرعت	۲
۳	۱-۲ تعاریف	۳
۳	۱-۱-۲ مدل‌های مولد	۳
۳	۲-۱-۲ مدل‌های پخشی	۳
۵	۳-۱-۲ مدل‌های پخشی احتمالاتی نوپزدا	۵
۵	۱-۳-۱-۲ تعریف مدل	۵
۵	۲-۳-۱-۲ مسیر رو به جلو	۵
۶	۳-۳-۱-۲ مسیر رو به عقب	۶
۷	۴-۳-۱-۲ تابع مدل مولد	۷
۷	۵-۳-۱-۲ تعریف تابع خطا	۷
۸	۶-۳-۱-۲ تغییر پارامتر تابع خطا	۸
۹	۷-۳-۱-۲ فرآیند یادگیری	۹
۱۰	۸-۳-۱-۲ فرآیند تولید نمونه‌ی جدید	۱۰
۱۰	۴-۱-۲ انواع معماری مدل‌های پخشی نوپزدا ی احتمالاتی	۱۰
۱۱	۱-۴-۱-۲ مدل پخشی نهان	۱۱
۱۳	۲-۴-۱-۲ مدل پخشی مبتنی بر مبدل	۱۳
۱۴	۲-۲ رویکردهای افزایش سرعت	۱۴
۱۶	۱-۲-۲ رویکردهای کاهش تعداد گام‌ها	۱۶
۱۷	۲-۲-۲ رویکردهای کاهش محاسبات هر گام	۱۷
۱۸	۱-۲-۲-۲ گسسته‌سازی	۱۸
۱۸	۲-۲-۲-۲ کاهش توکن‌ها	۱۸
۱۹	۱-۲-۲-۲-۲ روش ادغام توکن‌ها	۱۹
۲۰	۲-۲-۲-۲-۲ روش همجوشی توکن‌ها	۲۰
۲۳	۳-۲-۲-۲ استفاده از حافظه‌ی پنهان	۲۳
۲۳	۱-۳-۲-۲-۲ روش استفاده از حافظه‌ی پنهان عمیق	۲۳
۲۵	۲-۳-۲-۲-۲ مدل پخشی سریع‌تر	۲۵
۲۷	۳-۳-۲-۲-۲ رویکرد گیت T	۲۷
۳۰	۳ روش پیشنهادی	۳۰
۳۱	۱-۳ روش پایه	۳۱
۳۳	۲-۳ انطباقی کردن روش ادغام توکن‌ها	۳۳

۳-۳	استفاده از حافظه‌ی نهان در ادغام توکن‌ها	۳۴
۴	آزمایش‌ها و نتایج	۳۷
۱-۴	مجموعه داده	۳۸
۱-۱-۴	مجموعه داده‌ی <i>ImageNet</i>	۳۸
۲-۱-۴	مجموعه داده‌ی <i>COCO</i>	۳۸
۲-۴	معیارهای ارزیابی	۳۹
۳-۴	تعیین ابرپارامترها	۴۱
۴-۴	تحلیل و بررسی نتایج	۴۲
	کتاب‌نامه	۴۴
	واژه‌نامه‌ی فارسی به انگلیسی	۴۸
	واژه‌نامه‌ی انگلیسی به فارسی	۵۱

شکل	فهرست تصاویر	صفحه
۱-۲	روند کلی یک مدل پخشی	۴
۲-۲	فرآیند پخش در مپناها	۵
۳-۲	معماری $pixelCNN++$	۱۱
۴-۲	معماری شبکه‌ی یو	۱۲
۵-۲	معماری مدل‌های پخشی پنهان	۱۳
۶-۲	معماری مدل‌های مبتنی بر مبدل	۱۴
۷-۲	مشکل سه‌گانه‌ی مدل‌های مولد	۱۶
۸-۲	رویکردهای انتخاب توکن‌های مبدا و مقصد در ادغام توکن‌ها	۲۰
۹-۲	روال کار ساخت گراف دوبخشی و ادغام توکن‌ها	۲۱
۱۰-۲	محل قرارگیری بلوک‌های ادغام و رفع ادغام در بلوک مبدل در مدل‌های پخشی	۲۱
۱۱-۲	مشاهدات تجربی مبنی بر وجود افزونگی زمانی در ویژگی‌های سطح بالا در مدل‌های پخشی	۲۴
۱۲-۲	کلیت روش استفاده از حافظه‌ی پنهان عمیق	۲۵
۱۳-۲	مشاهدات تجربی مبتنی بر تغییرات ناچیز کدگذار شبکه‌ی یوی مدل پخشی	۲۶
۱۴-۲	کلیت مدل پخشی سریع‌تر	۲۷
۱۵-۲	نمودار تفاضل تغییرات بلوک توجه متقابل در گام‌های زمانی متوالی	۲۸
۱۶-۲	تقسیم فرآیند نويززدایی به دو بخش برنامه‌ریزی معنایی و افزایش کیفیت	۲۹
۱-۳	تصویر نمادین تفاوت حالت انطباق‌پذیر و حالت ثابت ادغام توکن‌ها	۳۴
۲-۳	نمودار فاصله‌ی جاکارد مجموعه‌ی جفت‌های ادغام در روش ادغام توکن‌ها	۳۵
۳-۳	شمای کلی روش پیشنهادی و استفاده از حافظه‌ی پنهان در فرآیند ادغام توکن‌ها	۳۶

صفحه	فهرست جداول	جدول
۳۱ . . .	۱-۳ نتایج اعمال روش ادغام توکن‌ها بر بلوک‌های مبدل لایه‌های مختلف شبکه‌ی یو	
۴۱	۱-۴ معیارهای کیفیت و زمان برای آستانه‌های مختلف در روش ادغام تطبیقی	
	۲-۴ گام‌های محاسبه در تنظیمات مختلف در نظر گرفته شده در روش ادغام توکن تطبیقی	
۴۲	مبتنی بر حافظه‌ی پنهان	
	۳-۴ عملکرد کلی روش ادغام توکن مبتنی بر حافظه‌ی پنهان در تنظیمات گام‌های محاسبه‌ی	
۴۲	متفاوت	
۴۳ .	۴-۴ مقایسه‌ی مدل پایه و مدل ادغام توکن‌ها با روش پیشنهادی در بهترین تنظیمات آن	

فهرست اختصارات

عنوان اختصاری عنوان کامل

مپنا مدل پخشی نوپززدای احتمالاتی

فصل اول

مقدمه

فصل دوم

مدل‌های پخشی و رویکردهای افزایش سرعت

۱-۲ تعاریف

۱-۱-۲ مدل‌های مولد

مدل‌های مولد عمیق دسته‌ای از مدل‌های پرکاربرد و جدید در حوزه‌ی هوش مصنوعی هستند که تلاش می‌کنند تا توزیع اصلی داده‌ی نمونه‌برداری شده را مدل کنند و با استفاده از این توزیع انواع مختلفی از سوالات در خصوص داده و دسته‌های آن را پاسخ دهند. در واقع اصلی‌ترین هدف یک مدل مولد، استفاده از مشاهدات شبیه‌سازی شده یا نمونه برداری شده از یک توزیع برای تخمین آن توزیع و سپس تولید نمونه‌های جدید از توزیع تخمین زده شده می‌باشد. مدل‌های مولد کاربردهای متنوعی در یادگیری ماشین دارند، از جمله‌ی این کاربردها می‌توان به تولید عکس‌های باکیفیت، تولید قطعه‌های موسیقی و یا صوت، افزایش کارایی یادگیری نیمه نظارتی^۱، تشخیص ناهنجاری^۲، یادگیری تقلیدی و کمک به یادگیری تقویتی اشاره کرد. پیشرفت‌های اخیر این حوزه دو رویکرد کلی را دنبال می‌کنند: رویکردهای مبتنی بر درست‌نمایی^۳ و شبکه‌های مولد تقابلی^۴. در رویکردهای مبتنی بر درست‌نمایی از لگاریتم درست‌نمایی یا خطاهای جایگزین به عنوان تابع هدف یادگیری استفاده می‌شود؛ در حالیکه در شبکه‌های مولد تقابلی تلاش می‌شود تا یادگیری با یک رویکرد تقابلی بین دو شبکه‌ی تمایزگر^۵ و تولیدکننده^۶ صورت گیرد. با وجود عملکرد قابل توجه مدل‌های مبتنی بر درست‌نمایی و شبکه‌های مولد تقابلی، این مدل‌ها به واسطه‌ی نوع تعریفشان درگیر برخی محدودیت‌های ذاتی هستند. به عنوان مثال مدل‌های مبتنی بر درست‌نمایی برای ساخت توزیع‌های نرمال شده نیازمند به کارگیری معماری‌های خاص یا استفاده از توابع خطای جایگزین (مانند حد پایین مشاهده^۷ در خودکدگذارهای تغییراتی^۸) در فرآیند یادگیری هستند. شبکه‌های مولد تقابلی مشکلات موجود در مدل‌های مبتنی بر درست‌نمایی را ندارند اما فرآیند یادگیری آنها به خاطر ماهیت تقابلی آن می‌تواند دچار ناپایداری شود. همچنین تابع هدف مدل‌های مولد تقابلی برای مقایسه و ارزیابی انواع این مدل‌ها چندان مناسب نیست [۲۴].

۲-۱-۲ مدل‌های پخشی

مدل‌های پخشی^۹ یک دسته‌ی نوظهور از مدل‌های مولد هستند که با توجه به عملکرد مناسب آنها در حوزه‌های تصویر، صوت، متن و گراف به سرعت مورد استقبال قرار گرفتند. وجه مشترک همه‌ی مدل‌های

^۱Semi-supervised learning^۲Anomaly^۳Likelihood^۴Generative adversarial networks^۵Discriminator^۶Generator^۷Evidence lower bound^۸Variational autoencoders^۹Diffusion models

پخشی این است که در یک فرآیند رو به جلو^{۱۰} داده را با نویز تخریب می‌کنند و در نتیجه از توزیع داده به یک توزیع دلخواه قابل محاسبه می‌رسند و سپس در یک مسیر رو به عقب که توسط مدل یاد گرفته می‌شود، تلاش می‌کنند تا داده را بازسازی و از یک نمونه از توزیع دلخواه به یک نمونه از توزیع داده‌ها برسند. این موضوع در شکل ۱-۲ نشان داده شده است.



شکل ۱-۲: روند کلی یک مدل پخشی

در هر مدل پخشی یک فرآیند ثابت گام به گام رو به جلو برای تخریب داده و یک فرآیند معکوس برای تولید یک نمونه داده از نویز وجود دارد [۱۰].

با وجود اینکه چهارچوب گفته شده در بند قبل در همه‌ی مدل‌های پخشی وجود دارد؛ رویکردهای مختلفی برای فرموله سازی و پیاده سازی این روش وجود دارد. در مدل‌های مولد مبتنی بر تابع امتیاز^{۱۱} تلاش می‌شود با استفاده از گرادینان لگاریتم توزیع داده، که به آن تابع امتیاز^{۱۲} می‌گوییم، و یک فرآیند گام به گام حرکت در جهت این گرادینان نمونه‌ی جدید تولید کنیم. از طرفی در مدل‌های پخشی نویززدای احتمالاتی فرآیند تخریب داده و تولید نمونه با یک زنجیره‌ی مارکوفی احتمالاتی مدل می‌شود و در نهایت در مدل‌های پخشی مبتنی بر معادلات دیفرانسیل تصادفی، فرآیند تخریب به صورت یک معادله دیفرانسیل تصادفی پیوسته مدل می‌شود که با پیدا کردن معکوس این معادله و گسسته سازی آن امکان تولید نمونه‌ی جدید از توزیع داده‌ها فراهم می‌گردد.

می‌توان نشان داد که مدل‌های پخشی بسیاری از محدودیت‌های مدل‌های مولد دیگر مانند نیاز به تمایزگر در شبکه‌های مولد تقابلی، نیاز به مقیدسازی شبکه در جریان‌های نرمال‌ساز^{۱۳} و نیاز به تنظیم توزیع‌های پسین در خودکدگذارهای تغییراتی را ندارند و قابلیت تولید تصاویر باکیفیت و متنوع را دارند. مشکل اصلی در مدل‌های پخشی فرآیند نمونه‌برداری طولانی آنهاست که باعث عدم کارایی این مدل‌ها در کاربردهای تعاملی می‌شود. با رفع این موضوع می‌توان اطمینان حاصل کرد که این دسته از مدل‌های مولد از توانایی همه‌جانبه‌ای برای به کارگیری در حوزه‌های مختلف برخوردار هستند.

¹⁰Forward process

¹¹Score-based generative models

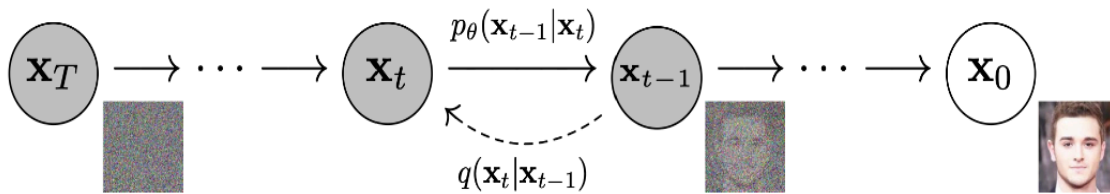
¹²Score function

¹³Normalizing flows

۳-۱-۲ مدل‌های پخشی احتمالاتی نويززدا

۱-۳-۱-۲ تعريف مدل

مدل‌های پخشی نويززداي احتمالاتي^{۱۴} يا به اختصار مپناها مدل‌هایی مبتنی بر متغیر پنهان^{۱۵} هستند که از یک زنجیره‌ی مارکوفی^{۱۶} برای فرآیند پخشی استفاده می‌کنند. رویکرد کلی این مدل‌ها به این صورت است که در یک مسیر رو به جلو هر توزیع پیچیده‌ای از داده‌ها را می‌گیرد و آن را به یک توزیع اولیه که ساده و قابل محاسبه است تبدیل می‌کند و سپس یک مسیر روبه عقب محدود به زمان را یاد می‌گیرد که نمونه‌هایی از توزیع اولیه را به نمونه‌های توزیع داده تبدیل کند و این همان ایده‌ی کلی مدل مولد پخشی می‌باشد. تصویر ۲-۲ این فرآیند را نشان می‌دهد. بنابراین این مدل سه مشخصه‌ی اصلی دارد: مسیر رو به جلو، مسیر رو به عقب و تابع مدل مولد که در ادامه هر یک را توضیح می‌دهیم.



شکل ۲-۲: فرآیند پخش در مپناها

مدل گراف‌ی جهت‌دار یک مپنا که به صورت یک زنجیره‌ی مارکوف روبه جلو و معکوس آن می‌باشد [۸].

۲-۳-۱-۲ مسیر رو به جلو

اگر توزیع داده را با $q(x^{(0)})$ نشان دهیم، این توزیع به صورت مرحله به مرحله به یک توزیع $\pi(y)$ که از نظر آنالیزی قابل بیان است تبدیل می‌شود و این تبدیل توسط یک هسته‌ی $T_\pi(y|y'; \beta)$ صورت می‌گیرد که β را نرخ پخش^{۱۷} می‌گوییم و داریم:

$$\pi(y) = \int dy' T_\pi(y|y'; \beta) \pi(y') \quad (۱-۲)$$

^{۱۴}Denoising diffusion probabilistic models(DDPMs)

^{۱۵}Latent variable models

^{۱۶}Markov Chain

^{۱۷}Diffusion rate

$$q(x^{(t)}|x^{(t-1)}) = T_{\pi}(x^{(t)}|x^{(t-1)}; \beta_t) \quad (2-2)$$

و بنابراین مسیر رو به جلو^{۱۸} با شروع از توزیع داده‌ها و طی T گام پخشی به صورت زیر است:

$$q(x^{(0...T)}) = q(x^{(0)}) \prod_{t=1}^T q(x^{(t)}|x^{(t-1)}) \quad (3-2)$$

در ادامه‌ی این فصل هسته‌ی $q(x^{(t)}|x^{(t-1)})$ هسته‌ی گاوسی در نظر گرفته می‌شود اما هسته‌های دیگری چون هسته‌ی دوجمله‌ای^{۱۹} نیز وجود دارند. یک ویژگی حائز اهمیت مسیر رو به جلو این است که می‌توان به جای استفاده از فرم زنجیره‌ی مارکوفی آن از یک فرم بسته استفاده کرد به این معنا که برای یافتن $x^{(t)}$ در زمان دلخواه t می‌توان از رابطه‌ی زیر کمک گرفت:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I) \quad (4-2)$$

که $\alpha_t := 1 - \beta_t$ و $\bar{\alpha}_t := \prod_{s=1}^t \alpha_s$ تعریف می‌شود.

۳-۳-۱-۲ مسیر رو به عقب

برای یادگیری توزیع مولد نیاز است که بتوانیم برعکس مسیر رو به جلو را یاد بگیریم و این به این معناست که باید بتوانیم از یک توزیع دلخواه قابل محاسبه به توزیع داده برسیم. بنابراین با تعریف توزیع اولیه به صورت $p(x^{(T)}) = \pi(x^{(T)})$ داریم:

$$p(x^{(0...T)}) = p(x^{(T)}) \prod_{t=1}^T p(x^{(t-1)}|x^{(t)}) \quad (5-2)$$

می‌توان نشان داد که اگر فرآیند پخش گاوسی باشد و گام‌های پخش β خیلی کوچک باشند آنگاه معکوس فرآیند پخش نیز گاوسی می‌باشد [۴]. و بنابراین هر چقدر که طول این زنجیره بیشتر باشد β کوچکتر می‌شود و می‌توان از تابع گاوسی برای مسیر معکوس نیز کمک گرفت. در فرآیند یادگیری با توجه به

¹⁸Forward trajectory

¹⁹Binomial kernel

اینکه هسته‌ی گذر^{۲۰} زنجیره‌ی مارکوفی یک تابع گاوسی است، برای ساخت زنجیره تنها نیاز به میانگین و کوواریانس این تابع گاوسی داریم و در ادامه سعی می‌کنیم آنها را تخمین بزنیم.

۴-۳-۱-۲ تابع مدل مولد

تابع احتمال تولید شده توسط مپنا به صورت زیر می‌باشد:

$$p(x^{(0)}) = \int dx^{(1...T)} p(x^{(0...T)}) \quad (۶-۲)$$

از آنجاییکه انتگرال در حالت کلی غیرقابل محاسبه^{۲۱} است، می‌توان با ایده گرفتن از نمونه برداری اهمیت^{۲۲} به جای محاسبه‌ی انتگرال از میانگین احتمال نسبی مسیر رو به جلو و مسیر رو به عقب استفاده کرد که به صورت زیر قابل بیان است:

$$p(x^{(0)}) = \int dx^{(1...T)} q(x^{(1...T)}|x^{(0)}) \prod_{t=1}^T \frac{p(x^{(t-1)}|x^{(t)})}{q(x^{(t)}|x^{(t-1)})} \quad (۷-۲)$$

۵-۳-۱-۲ تعریف تابع خطا

با توجه به مدل تعریف شده در قسمت قبل، آموزش شبکه معادل بیشینه کردن لگاریتم درست‌نمایی^{۲۳} تابع مدل مولد و یا کمینه کردن منفی این تابع است. بنابراین تابع هزینه را می‌توان به صورت زیر نوشت:

$$L = \int dx^{(0)} q(x^{(0)}) \log p(x^{(0)}) \quad (۸-۲)$$

در اینجا برای آموزش به جای بهینه سازی مستقیم لگاریتم درست‌نمایی از بهینه سازی حد تغییراتی^{۲۴} آن کمک می‌گیریم که به صورت زیر قابل تعریف است:

$$\mathbb{E}[-\log p_{\theta}(x^0)] \leq \mathbb{E}_q[-\log \frac{p_{\theta}(x^{0:T})}{q(x^{1:T}|x^0)}] = \mathbb{E}_q[-\log p(x^T) - \sum_{t \geq 1} \log \frac{p_{\theta}(x^{t-1}|x^t)}{q(x^t|x^{t-1})}] =: L \quad (۹-۲)$$

²⁰Transition kernel

²¹Intractable

²²Importance sampling

²³Log-likelihood

²⁴Variational bound

۲-۱-۳-۶ تغییر پارامتر تابع خطا

از آنجایی که ترکیب مسیر رو به جلو و مسیر معکوس شبیه به یک کدگذار تغییراتی است، می‌توان تابع هزینه را به صورت زیر بازنویسی کرد:

$$\mathcal{L}_{vib} = \mathcal{L}_0 + \mathcal{L}_1 + \dots + \mathcal{L}_{T-1} + \mathcal{L}_T \quad (10-2)$$

$$\mathcal{L}_0 = -\log p_\theta(x_0|x_1) \quad (11-2)$$

$$\mathcal{L}_{t-1} = \mathcal{D}_{KL}(q(x_{t-1}|x_t, x_0) \| p_\theta(x_{t-1}|x_t)) \quad (12-2)$$

$$\mathcal{L}_T = \mathcal{D}_{KL}(q(x_T|x_0) \| p(x_T)) \quad (13-2)$$

و از آنجایی که در صورت کوچک بودن β_t هر دوی $q(x_{t-1}|x_t, x_0)$ و $p_\theta(x_{t-1}|x_t)$ توزیع‌هایی گاوسی هستند در نتیجه دیورجنس KL ^{۲۵} آنها را می‌توان به فرم تفاضل میانگین آنها در نظر گرفت و در نتیجه برای معادله ی (۱۲-۲) داریم:

$$\mathcal{L}_{t-1} = \mathbb{E}_q \left[\frac{1}{2\sigma_t^2} \|\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\|^2 \right] + C \quad (14-2)$$

که در واقع نشان می‌دهد که در فرآیند آموزش یک مدل پخشی بهینه سازی خطا شامل تخمین میانگین پسین^{۲۶} فرآیند رو به جلو می‌باشد. لازم است اشاره کنیم که در مقاله‌ی اصلی مقدار واریانس ثابت در نظر گرفته شده است. از طرفی با تغییر پارامتر $x_t(x_0, \epsilon) = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{(1 - \bar{\alpha}_t)}\epsilon$ برای $\epsilon \sim \mathcal{N}(0, I)$ و قراردادی آن در فرمول معادله‌ی (۱۴-۲) می‌توان نشان داد که تخمین $\mu_\theta(x_t, t)$ به صورت زیر قابل پارامتری سازی است:

²⁵KL-divergence

²⁶Posterior mean

$$\mu_{\theta}(x_t, t) = \tilde{\mu}_t \left(x_t, \frac{1}{\sqrt{\bar{\alpha}_t}} (x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_{\theta}(x_t)) \right) = \frac{1}{\sqrt{\bar{\alpha}_t}} (x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t)) \quad (15-2)$$

که ϵ_{θ} یک تخمین از نویز موجود در x_t می‌باشد و در نتیجه برای محاسبه ی یک نمونه ی x_{t-1} تنها کافیست که عبارت زیر را محاسبه کنیم:

$$x_{t-1} = \frac{1}{\sqrt{\bar{\alpha}_t}} (x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) + \sigma_t z) \quad (16-2)$$

که z از توزیع گاوسی با میانگین صفر و کوواریانس همانی می‌آید. این تغییر پارامتر نشان می‌دهد که برای آموزش یک فرآیند رو به عقب می‌توان از تخمین $\tilde{\mu}_t$ با تابع میانگین وابسته به زمان μ_{θ} استفاده کرد و یا مستقیماً نویز اضافه شده به داده را با تخمین ϵ محاسبه کرد.

۷-۳-۱-۲ فرآیند یادگیری

با توجه به تغییر پارامتر انجام شده در قسمت قبل و همچنین به کارگیری آن در معادله ی (۱۰-۲) می‌توان به تابع هزینه ای رسید که کاملاً نسبت به θ مشتق پذیر است و برای آموزش مناسب است اما برای سادگی پیاده سازی و افزایش کیفیت نمونه‌های تولیدی می‌توان تابع هزینه را به شکل زیر نیز بازنویسی کرد:

$$L_{simple}(\theta) = \mathbb{E}_{t, x_0, \epsilon} \left[\left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2 \right] \quad (17-2)$$

که t از یک توزیع یونیفرم بین ۱ و T می‌آید و واریانس β_t در این فرآیند یک پارامتر ثابت بدون یادگیری می‌باشد. الگوریتم ۱ فرآیند آموزش یک مپنا را نشان می‌دهد.

الگوریتم ۱ آموزش

۱: مراحل زیر را تکرار کن

۲: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$

۳: $t \sim \text{Uniform}(\{1, \dots, T\})$

۴: $\epsilon \sim \mathcal{N}(0, \mathbf{I})$

۵: گام نزول در جهت گرادیان را روی تابع زیر اعمال کن

$$\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2$$

۶: تا زمانی که همگرا شود

۲-۱-۳-۸ فرآیند تولید نمونه‌ی جدید

همانطور که در الگوریتم ۲ نشان داده شده است؛ برای نمونه برداری ابتدا یک نمونه از توزیع $\mathcal{N}(0, \mathbf{I})$ تولید می‌کنیم سپس به صورت معکوس هر بار نمونه را به شبکه می‌دهیم و نمونه‌ی نویززدایی شده را دریافت می‌کنیم. این فرآیند تا جایی ادامه پیدا می‌کند که به یک نمونه از توزیع $q(\mathbf{x}_0)$ برسیم.

الگوریتم ۲ نمونه‌برداری

۱: $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$

۲: برای $t = T, \dots, 1$ مراحل زیر را انجام بده:

۳: $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ اگر $t > 1$ ، در غیر این صورت $\mathbf{z} = 0$

۴: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$

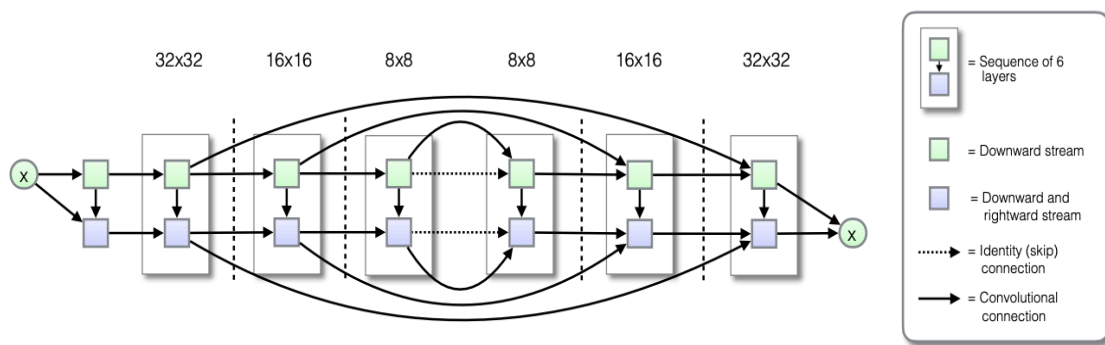
۵: پایان حلقه

۶: مقدار $\mathbf{x}_0$ را برگردان.

در بخش بعدی سراغ انواع معماری‌ها و پیاده‌سازی‌های مپناها می‌رویم.

۲-۱-۴ انواع معماری مدل‌های پخشی نویززدای احتمالاتی

در ساختار مپناها از هسته‌ی اصلی $++ \text{pixelCNN}$ [۲۱] استفاده شده است که در واقع نوعی شبکه‌ی یو با اتصالات گسترده‌ی باقی‌مانده‌ای می‌باشد. شکل ۲-۳ ساختار کلی شبکه‌ی به کار گرفته شده را نشان می‌دهد.



شکل ۲-۳: معماری $pixelCNN++$

شبکه‌ی $pixelCNN++$ که نوعی شبکه‌ی یو با اتصالات باقی‌مانده‌ای گسترده است به عنوان شبکه‌ی اصلی در مپ‌ها استفاده می‌شود [۲۱].

تا قبل از ارائه‌ی مدل پخشی نهان [۱۹]^{۲۷} عمده‌ی ساختار ارائه شده برای مپ‌ها به تنهایی بر پایه‌ی شبکه‌ی یو [۲۰] یا یکی از انواع این شبکه بود. شبکه‌ی یو یک ساختار کدگذار-کدگشا^{۲۸} است که ابتدا برای مسالهی قطعه‌بندی تصاویر پزشکی ارائه شد. ویژگی خاص این شبکه نسبت به سایر ساختارهای کدگذار-کدگشا وجود اتصالات پرشی^{۲۹} می‌باشد. این اتصالات با هدف ترکیب اطلاعات سطح بالا و اطلاعات محلی و نیز افزایش رزولوشن تصویر خروجی استفاده شدند. شکل ۲-۴ ساختار یک شبکه‌ی یو را نشان می‌دهد.

۲-۴-۱-۲ مدل پخشی نهان

استفاده از شبکه‌ی یو به تنهایی، چالش‌های زیاد دارد؛ از جمله‌ی این مشکلات می‌توان به هزینه‌ی محاسباتی و سنگین بودن شبکه‌ی یو اشاره کرد که این موضوع با افزایش رزولوشن تصویر می‌تواند هزینه‌ی زیادی را ایجاد کند. از طرفی در اکثر مپ‌ها فرآیند آموزش و نمونه‌برداری شبکه‌ی یو در فضای پیکسلی انجام می‌شود که ما را نیازمند تعریف یک شبکه‌ی متناظر به ازای هر رزولوشن و یا تغییر ابعاد تصویر می‌کند. علاوه بر موارد ذکر شده در شبکه‌های یو به کار گرفته شده در فرآیند مپ‌ها امکان شرطی‌سازی انعطاف پذیر وجود ندارد. این شبکه امکان انجام شرطی‌سازی بر اساس نام کلاس‌ها یا نویز ابتدایی تصویر را دارد ولی امکان دریافت متن و یا نقشه‌ی معنایی^{۳۰} و پردازش آنها به عنوان شرط را بدون بهینه‌سازی در معماری ندارد.

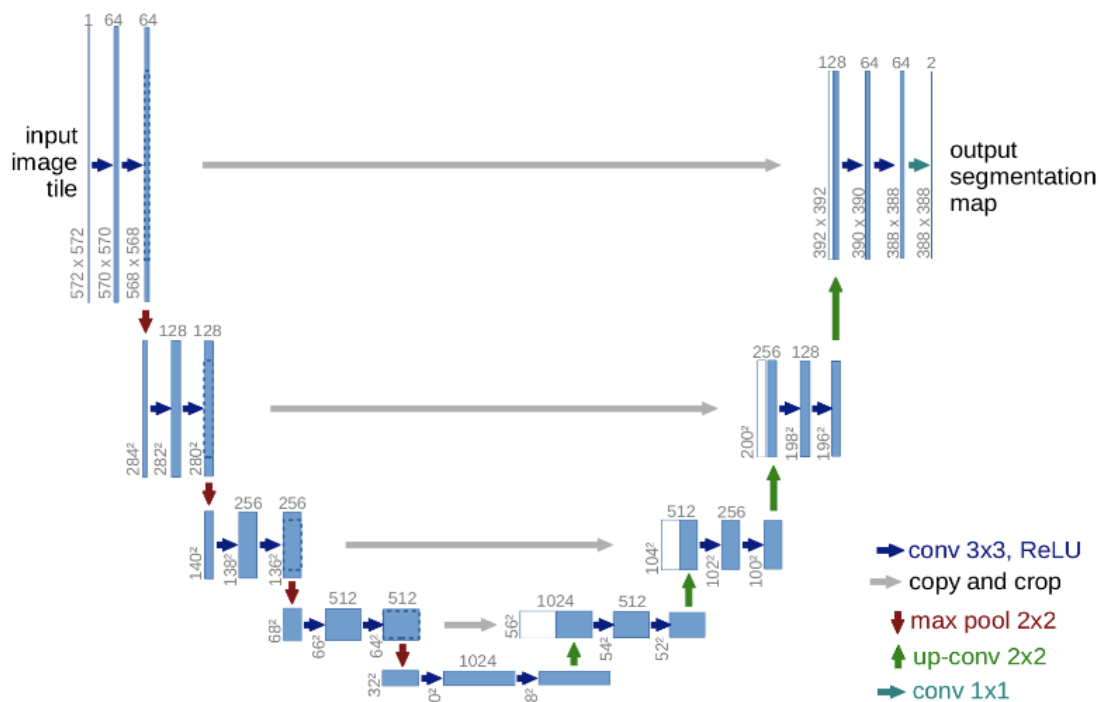
مدل‌های پخشی نهان با تمرکز بر رفع نقاط ضعف اشاره شده در مدل‌های شبکه‌ی یو ابتدایی ارائه شدند. این مدل‌ها تلاش می‌کنند که اولاً با انتقال تصویر از فضای پیکسلی به فضای پنهان با ابعاد کمتر از حجم

²⁷Latent Diffusion model

²⁸Encoder-Decoder

²⁹Skip Connection

³⁰Semantic Map



شکل ۲-۴: معماری شبکه‌ی یو

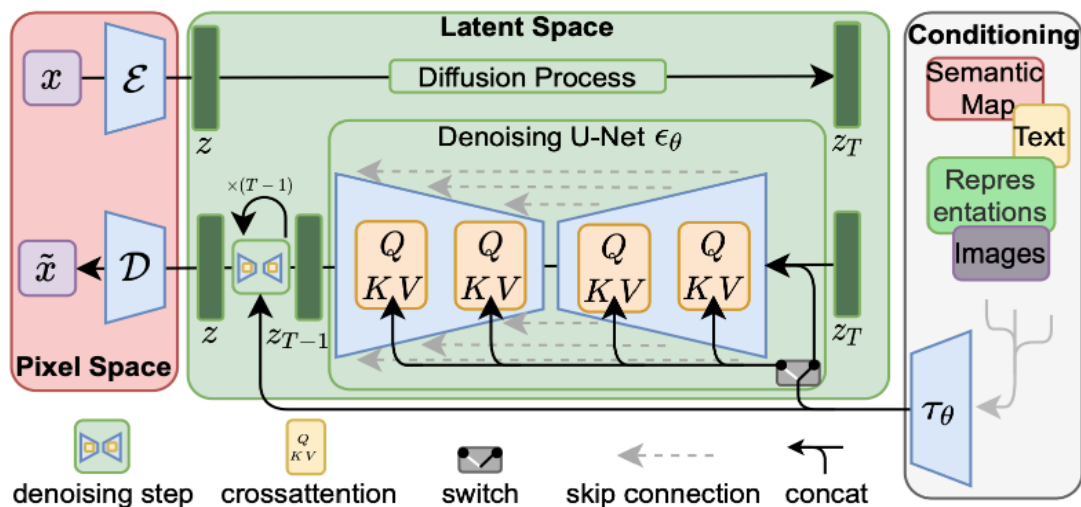
معماری کلی شبکه‌ی یو که اولین بار برای مساله‌ی قطعه‌بندی معنایی تصاویر پزشکی ارائه شد و شامل سه بخش کدگذار، کدگشا و اتصالات پرشی است [۲۰].

محاسبات بکاهند. و ثانياً با اضافه کردن بلوک‌های توجه متقابل^{۳۱} انعطاف پذیری مدل را به واسطه‌ی امکان شرطی‌سازی بر روی هر شکلی از شرط شامل متن یا نقشه‌ی معنایی افزایش دهند. خروجی این تغییرات باعث تولید تصاویر با کیفیت بهتر و در رزولوشن‌های بالاتر می‌باشد.

شکل ۲-۶ معماری این مدل را نشان می‌دهد. این مدل با به کارگیری یک خودکدگذار تغییراتی از پیش آموزش داده شده، ابتدا تصویر را از فضای پیکسلی به فضای پنهان می‌برد. در فضای پنهان، فرآیند یادگیری و نمونه‌برداری مشابه هر مدل مپنای دیگری انجام می‌شود و معماری اصلی شبکه‌ی نويززدا نیز یک شبکه‌ی یو با اتصالات پرشی و بلوک‌های توجه متقابل است. این مدل با استفاده از بلوک‌های توجه متقابل شرط را روی فرآیند نويززدایی اعمال می‌کند و در نهایت این مدل پس از پایان مراحل نويز زدایی تصویر را از فضای پنهان به فضای پیکسلی باز می‌گرداند.

مدل‌های *Stable Diffusion* که در این پروژه به کار گرفته شده‌اند؛ از این معماری تبعیت می‌کنند. که در فصل‌های پیش رو در خصوص جزئیات پیاده‌سازی آنها بیشتر صحبت خواهیم کرد.

³¹Cross-Attention Blocks



شکل ۲-۵: معماری مدل‌های پخشی پنهان

در شکل بالا شبکه‌ی ϵ کدگذار خود کدگذار تغییراتی است که تصویر ورودی x را دریافت می‌کند و آن را به تصویر در فضای پنهان z تبدیل می‌کند. همچنین τ_θ یک کدگذار مختص دامنه‌ی مساله^{۳۲} است که برای تبدیل شرط به یک بازنمایی قابل درک توسط مدل به کار گرفته می‌شود. همچنین D نیز کدگشای خود کدگذار تغییراتی است که تصویر نویززدایی شده در فضای پنهان را به تصویر در فضای پیکسلی می‌برد^[۱۹].

۲-۴-۱-۲ مدل پخشی مبتنی بر مبدل

در مدل‌های پخشی مبتنی بر مبدل^{۳۳} [۱۸] به جای استفاده از شبکه‌ی یو به عنوان شبکه‌ی نویززا از ساختار مبدل تصویری^{۳۴} استفاده می‌شود. انتخاب این مبدل‌ها به عنوان هسته‌ی اصلی مدل‌های پخشی از این جهت است که هم می‌توانند وابستگی‌های^{۳۵} قسمت‌های مختلف تصویر را مدل کنند و هم اینکه نسبت به شبکه‌ی یو ساختارهای مقیاس پذیرتری هستند. همچنین به کارگیری این مبدل‌ها در انواعی از دامنه‌ها مانند پردازش زبان طبیعی و سایر وظایف حوزه‌ی بینایی کارایی آنها را نسبت به سایر معماری‌ها نشان می‌دهد. و در نهایت وجود مکانیزم توجه به خود در مدل‌های مبدل باعث می‌شود که وابستگی‌های موجود در نواحی دور یا نزدیک تصویر به خوبی مدل می‌شوند.

این مدل، مشابه مدل پخشی پنهان در فضای پنهان حاصل از یک خودکدگذار تغییراتی نویززدایی می‌کند. این مدل به جای استفاده از یک شبکه‌ی نویززدایی مبتنی بر شبکه‌ی یو از یک ساختار مطابق شکل؟؟ استفاده می‌کند. در این حالت ابتدا تصویر ورودی به فضای پنهان برده می‌شود و در آنجا مدل تصویر پنهان را به تعدادی قطعه^{۳۶} تقسیم می‌کند که توسط مبدل پردازش می‌شوند. همچنین در این مدل

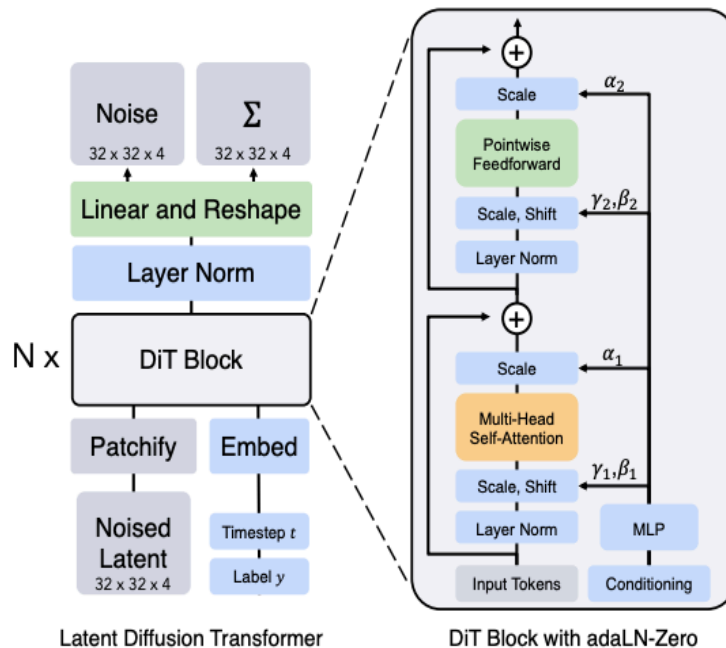
^{۳۳}Diffusion Transformer Models

^{۳۴}Vision Transformer

^{۳۵}Long-range Dependency

^{۳۶}Patch

از یک رویکرد به نام $adaLN - zero$ برای اضافه کردن نام کلاس و شرطی‌سازی استفاده می‌شود. این رویکرد برخلاف مدل‌های پخشی پنهان که از یک بلوک توجه متقابل استفاده می‌کند، ابتدا شرط یا بازنمایی آن را از یک شبکه‌ی تماماً متصل عبور می‌دهند و سپس آن را در قالب پارامترهای تغییر مقیاس^{۳۷} به لایه‌های نرمالسازی لایه‌ای اضافه می‌کند.



شکل ۲-۶: معماری مدل‌های مبتنی بر مبدل

همانطور که شکل نشان می‌دهد، ورودی‌های این مدل تصویر و زمان و برچسب کلاس هستند. تصویر ابتدا قطعه قطعه می‌شود و سپس وارد بلوک مبدل شده و در نهایت نیز یک تبدیل خطی روی آن انجام می‌شود تا با ترکیب قطعات مختلف تصویر نهایی ایجاد شود [۱۸].

این مدل بر اساس اندازه‌ی قطعه، تعداد سر^{۳۸}های مکانیزم توجه، تعداد لایه‌ها و اندازه‌ی لایه‌ی پنهان می‌تواند انواع گوناگونی داشته باشد. با توجه به آزمایشات انجام شده در مقاله مدل $DIT - XL$ با ۲۸ لایه، اندازه قطعه‌ی ۲، اندازه‌ی لایه‌ی پنهان ۱۱۵۲ و تعداد سرهای ۱۶ بهترین عملکرد را از خود ارائه می‌دهد.

۲-۲ رویکردهای افزایش سرعت

با وجود توسعه‌ی حجم زیادی از مدل‌های مولد در سال‌های اخیر برای انواعی از کاربردها از جمله تصویر، صدا، متن و گراف‌ها، همه‌ی مدل‌های موجود فعلی قابلیت ارضای سه نیاز اصلی یک مدل مولد به کار

^{۳۷}Scale Shift

^{۳۸}Head

گرفته شده در مسائل واقعی را ندارند [۲۵]. این سه نیاز اصلی عبارتند از:

۱. تولید نمونه‌های با کیفیت

۲. پوشش مد و تنوع نمونه‌های تولیدی

۳. نمونه برداری سریع و به صرفه از نظر محاسباتی

به عنوان مثال اکثر مدل‌های تولید تصویر فعلی روی تولید نمونه‌های با کیفیت تمرکز دارند اما تنوع داده‌ها نیز موضوع مهمی در کاهش تاثیرات اجتماعی منفی مدل‌های مولد است. همچنین برای کاربردهای تعاملی نیاز به نمونه برداری سریع یک مساله‌ی حائز اهمیت است. این چالش در ادبیات مدل‌ها مولد به عنوان مشکل سه‌گانه‌ی یادگیری مولد^{۳۹} شناخته می‌شود. هر کدام از مدل‌های مولد در حل بخشی از مشکلات یادگیری مولد تبحر دارند و بخش‌هایی از این سه‌گانه به عنوان ضعف آنها محسوب می‌شود. به عنوان مثال شبکه‌های مولد تقابلی قابلیت تولید نمونه‌های با کیفیت در زمان سریع را دارند اما پوشش مد ضعیفی دارند. از طرفی خودکدگذارهای تغییراتی به خوبی مدهای داده را پوشش می‌دهند و سرعت نمونه‌برداری مناسبی دارند اما کیفیت نمونه‌های تولیدی آنها کم است. برای مقایسه‌ی عملکرد مدل‌های پخشی با سایر مدل‌های مولد خوب است بدانیم که این مدل‌ها قابلیت رفع این مشکل سه‌گانه را دارند یا خیر. مدل‌های پخشی با ظهور خود توانستند از شبکه‌های مولد تقابلی در تولید نمونه‌های با کیفیت پیشی بگیرند و همچنین با تغییر و تنظیم این مدل‌ها و دستیابی به درست‌نمایی بهینه، قابلیت پوشش مدهای مختلف داده را دارند اما نمونه برداری از این مدل‌ها نیازمند ده‌ها بار استفاده از شبکه می‌باشد و همین امر سرعت نمونه برداری را کاهش می‌دهد و به کارگیری آنها در کاربرد را هزینه‌بر و دشوار می‌سازد. شکل ۲-۷ این مشکل سه‌گانه و حوزه‌ی عملکرد مناسب هر مدل را نشان می‌دهد.

با توجه به دسته‌بندی ذکر شده در بند قبل، اصلی‌ترین نقطه ضعف عمده‌ی مدل‌های پخشی سرعت نمونه‌برداری کم آنهاست و در همین راستا حجم زیادی از تحقیقات در سال‌های اخیر به کاهش زمان نمونه‌برداری مدل‌های پخشی اختصاص داده شده است. برای دسته‌بندی این تحقیقات هم رویکردهای متفاوتی وجود دارد. به عنوان مثال می‌توان روش‌ها را بر اساس نیازمندی به آموزش مجدد^{۴۰} و بدون نیاز به آموزش^{۴۱} به دو دسته تقسیم کرد.

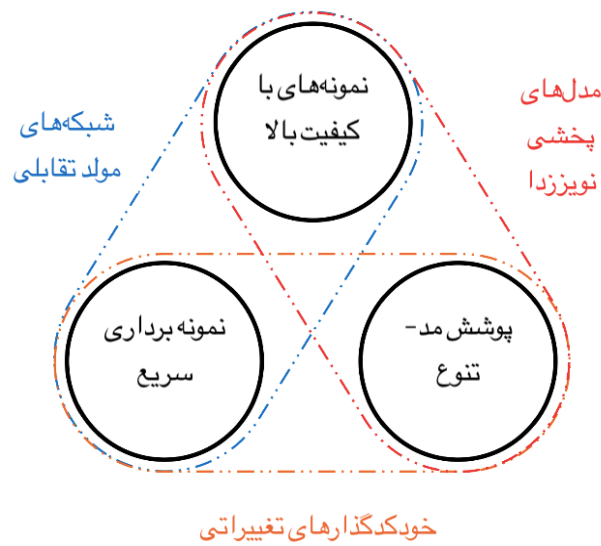
روش‌های مبتنی بر آموزش با تغییر قسمتی از مدل یا استفاده از رویکردهای تقطیر دانش^{۴۲} [۷] سعی بر کاهش زمان فرآیند نمونه‌برداری و استفاده از شبکه‌های کوچک‌تر و سریع‌تر برای نویززدایی دارند. مشکل عمده‌ی این روش‌ها این است که برای فرآیند آموزش مجدد نیاز به هزینه‌ی زمانی و پردازشی زیادی دارند و این موضوع با توجه به گسترش ابعاد تصاویر و سنگین‌تر شدن شبکه‌های نویززدا نیز رو به افزایش است. در مقابل روش‌های بدون نیاز به آموزش، از یک شبکه‌ی از پیش آموزش داده‌شده

^{۳۹}Generative learning trilemma

^{۴۰}Training-based Methods

^{۴۱}Training-Free Methods

^{۴۲}Knowledge Distillation



شکل ۲-۷: مشکل سه‌گانه‌ی مدل‌های مولد
سه نیاز اساسی در توسعه‌ی مدل‌های مولد و حوزه‌ی عملکرد موثر هر مدل [۲۵].

^{۴۳} استفاده می‌کنند و سعی می‌کنند تا با تغییر عناصر مختلف موجود در این مدل‌های از پیش آموزش داده شده، زمان نمونه برداری آنها را کاهش دهند. رویکردهای بهبود سرعت را می‌توان از جنبه‌های دیگری هم دسته‌بندی کرد، یکی از این جنبه‌ها محل اثر رویکرد بهبود سرعت است. اساساً مشکل زمانبر بودن فرآیند نمونه‌برداری در مدل‌های پخشی از دو مساله نشأت می‌گیرد. اول، تعداد گام‌های زیادی که برای نویززدایی نیاز است و دوم، سنگین بودن شبکه‌ی نویزدا که باعث زمانبر بودن محاسبات در هر گام به تنهایی می‌شود. بر همین اساس می‌توانیم رویکردهای افزایش سرعت را به دو دسته‌ی روش‌های کاهش تعداد گام و روش‌های کاهش محاسبات در هر گام تفکیک کنیم. در ادامه مختصراً به هر کدام می‌پردازیم.

۱-۲-۲ رویکردهای کاهش تعداد گام‌ها

رویکردهای موجود در این دسته تلاش می‌کنند تا با کاهش تعداد گام‌ها نویززدایی فرآیند نمونه‌برداری را تسریع کنند. رویکردهای این دسته را مجدد می‌توان به دو دسته‌ی نیازمند یادگیری و بدون نیاز به یادگیری تقسیم کرد. یک دسته‌ی شاخص در این رویکردها روش‌های مبتنی بر تقطیر دانش هستند. روش‌های تقطیر دانش با انتقال دانش از مدل‌های سنگین و نیازمند نمونه‌برداری کامل معلم به مدل‌های سریع‌تر دانش‌آموز تعداد گام مورد نیاز برای فرآیند نمونه‌برداری را کاهش می‌دهند. بر اساس [۱۷]

⁴³ Pre-trained Network

روش‌های تقطیر دانش نیز به سه دسته‌ی تقطیر دانش مبتنی بر توزیع^{۴۴}، تقطیر دانش مبتنی بر مسیر^{۴۵} و تقطیر دانش تقابلی^{۴۶} تقسیم می‌شوند. نقطه ضعف این مدل‌ها همانطور که در بحث رویکردهای نیازمند آموزش ذکر شد، فرآیند آموزش زمانبر و پرهزینه است و با توجه به اینکه عمدتاً هم شبکه‌ی معلم و هم شبکه‌ی دانش‌آموز بزرگ هستند؛ منابع پردازشی قابل توجهی برای انجام آنها نیاز است. رویکردهای دیگری که در دسته‌ی کاهش تعداد گام قرار می‌گیرند روش‌هایی هستند که با تغییر در قسمتی از فرآیند رو به جلو یا رو به عقب امکان پرش از روی چند گام را فراهم می‌کنند. از جمله‌ی این روش‌ها می‌توان به مدل‌های پخشی نویززدای ضمنی^{۴۷} [۲۳] اشاره کرد. این روش با تغییر فرآیند رو به جلو به یک فرآیند غیر مارکفی، این امکان را فراهم می‌کند که در فرآیند رو به عقب با تعداد گام کمتر و بدون از دست دادن کیفیت، فرآیند نمونه برداری را انجام دهیم. نقطه‌ی قوت اکثر روش‌هایی که در این دسته قرار می‌گیرند این است که بر مبنای یک تئوری ریاضیاتی بنا شده‌اند و در عین حال که انعطاف خوبی برای اعمال روی انواعی از معماری‌ها را دارند؛ می‌توانند به صورت عمود بر روش‌های کاهش محاسبات در هر گام بر مدل پخشی اعمال شوند تا حداکثر کاهش ممکن حاصل شود. از طرفی همانطور که در مثال‌های بالا هم مشاهده شد، عمده‌ی روش‌های شاخص در این دسته مبتنی بر آموزش هستند که نیاز به منابع پردازشی را به عنوان نقطه ضعف با خود دارند.

۲-۲-۲ رویکردهای کاهش محاسبات هر گام

روش‌های کاهش تعداد گام عمدتاً ساختار شبکه‌ی نویز زدا را به صورت یک جعبه سیاه در نظر می‌گیرند. این در حالیست که چه در ساختار شبکه‌ی یو و چه در ساختار مبتنی بر مبدل قسمت‌های متعددی وجود دارند که امکان بهینه‌سازی و بهبود عملکرد دارند. عمده‌ی رویکردهایی که در این قسمت معرفی می‌کنیم بر مبنای تغییر در معماری، فشردگی ساختار شبکه و تکرار محاسبات می‌باشند و هدف تمامی این تغییرات کاهش حجم محاسبات انجام شده در هر گام می‌باشد. رویکردهای کاهش محاسبات را می‌توان بر اساس محل اثر، هدف کلی و نیازمندی به آموزش به دسته‌های مختلفی تقسیم کرد. اما سه رویکرد شاخص، متاخر و عمدتاً بدون نیاز به آموزش در این دسته وجود دارند که بررسی آنها برای درک سازوکار رویکردهای کاهش محاسبات مفید خواهد بود و این دسته‌ها عبارتند از: گسسته‌سازی^{۴۸}، کاهش توکن‌ها^{۴۹} و استفاده از حافظه‌ی پنهان^{۵۰}.

⁴⁴Distribution-based KD

⁴⁵Trajectory-based KD

⁴⁶Adversarial KD

⁴⁷Denosing Diffusion Implicit Models(DDIM)

⁴⁸Quantization

⁴⁹Token Reduction

⁵⁰Caching

۱-۲-۲-۲ گسسته‌سازی

گسسته‌سازی یک تکنیک برای کاهش دقت مقادیر عددی موجود در یک مدل است که معمولاً با تبدیل اعداد ممیز شناور^{۵۱} ۳۲ یا ۱۶ بیتی به بازنمایی‌های کم‌دقت‌تر مانند اعداد صحیح ۸ بیتی انجام می‌شود. رویکردهای گسسته‌سازی به دو دسته‌ی آموزش آگاه از گسسته‌سازی^{۵۲} که در آنها فرآیند گسسته‌سازی همزمان با آموزش انجام می‌گیرد و گسسته‌سازی پس از آموزش^{۵۳} تقسیم می‌شوند.

در مدل‌های پخشی، فرآیند گسسته‌سازی باعث بهبود سرعت نمونه‌برداری و کاهش حجم شبکه‌ی نويززدا می‌شود و با توجه به فرآیند هزینه‌بر آموزش مدل‌های پخشی معمولاً در این مدل‌ها از روش‌های پس از آموزش برای گسسته‌سازی شبکه استفاده می‌شود. و این روش زمان و منابع محاسباتی کمتری نیاز دارد^[۲۲]. همچنین با توجه به ماهیت چندگامی بودن مدل‌های پخشی عمده‌ی روش‌های گسسته‌سازی که پیش از این برای شبکه‌های عمیق تک گامی گسترش داده شده بودند، روی این مدل باعث افت کیفیت می‌شوند. از این رو مدل‌هایی مانند گسسته‌سازی پس از آموزش *PTQ*^[۲۲] و گسسته‌سازی مدل پخشی^{۵۴} [۱۳] ارائه شدند که با تمرکز بر چندگامی بودن مدل‌های پخشی و ماهیت متفاوت داده‌ی دریافت شده در هر گام به گسسته‌سازی شبکه‌ی نويززدا می‌پردازند.

۲-۲-۲-۲ کاهش توکن‌ها

روش‌های کاهش توکن بر ورودی‌های مبدل‌ها اعمال می‌شوند و ایده‌ی کلی آنها این است که با توجه به اینکه هزینه‌ی محاسباتی مبدل‌ها بر حسب اندازه‌ی ورودی به صورت درجه دو است بنابراین کاهش اندازه‌ی ورودی هزینه‌ی محاسباتی و تاخیر زمانی موجود در مبدل‌ها را کاهش می‌دهد و آنها را بهینه‌تر می‌سازد. تعریف توکن در مدل‌های مختلف متفاوت است. در مدل‌های پخشی پنهان با توجه به اینکه ساختار اصلی شبکه‌ی یو است، توکن به تک تک پیکسل‌های تصویر در فضای پنهان گفته می‌شود. برخلاف مدل پخشی پنهان، در مدل‌های مبتنی بر مبدل با توجه به اینکه کلیت شبکه‌ی نويززدا از تعداد زیاد کدگذار مبدل ایجاد شده است می‌تواند به صورت قطعه‌های از تصویر تعریف شود. در ادامه‌ی این پژوهش و در مرور کارهای معرفی شده کلمه‌ی توکن معادل تعریف آن در مدل پخشی پنهان در نظر گرفته می‌شود.

رویکردهای کاهش توکن در انواعی از مسائل از جمله مدل‌های زبانی، مبدل تصویری^{۵۵} و انواع دیگری از مبدل‌ها استفاده شده است. به طور کلی رویکردهای کاهش توکن را می‌توان به سه دسته‌ی هرس توکن‌ها^{۵۶}، ادغام توکن‌ها^{۵۷} و روش‌های ترکیبی تقسیم کرد.

⁵¹Floating-point⁵²Quantization-aware Training⁵³Post-training Quantization⁵⁴Q-diffusion⁵⁵Vision Transformer(VIT)⁵⁶Token Pruning⁵⁷Token Merging

در هرس توکن‌ها یک مجموعه توکن به واسطه‌ی محاسبه‌ی یک تابع امتیاز بی اهمیت یا افزونه تشخیص داده می‌شوند و به طور کلی از محاسبات حذف می‌شوند. روش‌های هرس توکن می‌توانند به دو صورت متغیر^{۵۸} یا ثابت^{۵۹} اعمال شوند. در حالت ثابت همواره نرخ مشخصی از توکن‌ها دور ریخته می‌شوند اما در رویکرد متغیر بسته به امتیاز محاسبه شده، تصویر ورودی و سایر شاخص‌ها میزان توکن‌های مناسب برای هرس تعیین می‌شود. تعیین میزان هرس مساله‌ی مهمی است و هرس بیش از حد باعث از دست رفتن اطلاعات و خطا در مدل می‌شود.

روش‌های ادغام توکن‌ها به جای حذف توکن و کاهش اندازه‌ی ورودی مبدل از این طریق، توکن‌های مشابه را پیدا می‌کنند و آنها را با استفاده از یک روش ادغام ترکیب می‌کنند. این رویکردها نیز به دو دسته‌ی ادغام سخت^{۶۰} و ادغام نرم^{۶۱} تقسیم می‌شوند. در روش‌های سخت معمولاً از رویکردهای خوشه‌بندی برای تعیین توکن‌های قابل ادغام استفاده می‌شود در حالیکه در روش‌های نرم معمولاً از دوبخشی کردن توکن‌ها و محاسبه‌ی معیار شباهت استفاده می‌شود^[۵]. در روش‌های ترکیبی نیز عمدتاً از ترکیب ایده‌های هرس و ادغام توکن‌ها برای کاهش توکن استفاده می‌شود.

در معماری *SDv1.5* ۱۶ بلوک *BasicTransformer* در سطوح مختلف و بلوک‌های کدگذار و کدگشا استفاده شده است و با توجه به اینکه شبکه‌ی یوی نویز زدا از این مبدل و بلوک‌های باقی‌مانده‌ای تشکیل شده است، کاهش حجم محاسبات قابل انجام در یک بلوک مبدل می‌تواند به عنوان یکی از رویکردهای کاهش حجم محاسبات مدل در هر گام در نظر گرفته شود. در ادامه سه رویکرد پیاده‌سازی شده بر روی مدل‌های پخشی را معرفی می‌کنیم.

۲-۲-۲-۲ روش ادغام توکن‌ها

اکثر تصاویر از جمله آنهایی که در یک مدل پخشی تولید می‌شوند دارای افزونگی هستند و از آنجاییکه که مبدل تک تک توکن‌ها را پردازش می‌کند، حجمی از محاسبات غیر ضروری را به مدل تحمیل می‌کند. روش ادغام توکن‌ها^{۶۲} [۲] یک روش برای کاهش توکن‌های ورودی مبدل است. این روش ابتدا بر روی مبدل بینایی و سپس بر روی مدل پخشی پنهان *SDv1.5* پیاده‌سازی و اجرا شده است.

این روش با تقسیم توکن‌ها به دو دسته‌ی مبدا^{۶۳} و مقصد^{۶۴} و ساخت یک گراف دوبخشی از آنها روش ادغام را پیاده‌سازی می‌کند. برای دسته‌بندی هر توکن به عنوان مبدا یا مقصد رویکردهای مختلفی وجود دارد. شکل ۲-۸ انواعی از این حالات را نشان می‌دهد. انتخاب رندوم مبدا و مقصد باعث مشکلات زیادی می‌شود از جمله اینکه نواحی از تصویر می‌توانند هیچ توکن مقصدی نداشته باشند در نتیجه برای ادغام

⁵⁸Dynamic

⁵⁹Static

⁶⁰Hard-merging

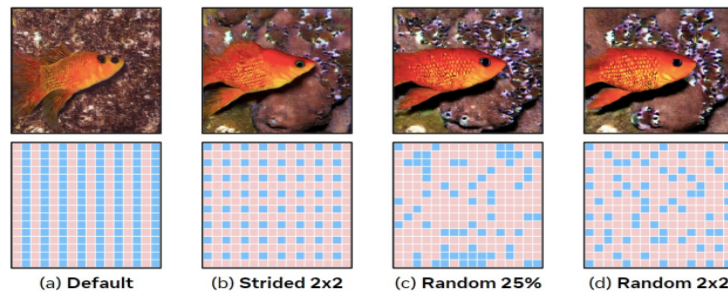
⁶¹Soft-merging

⁶²ToMe

⁶³Source

⁶⁴Destination

انتخاب نشوند در حالیکه قسمتی از پس زمینه یا نواحی کمتر متغییر بوده اند. بنابراین رویکرد مناسب‌تر در نظر گرفتن یک پنجره‌ی ۲ در ۲ در تصور و انتخاب یکی از عناصر آن به صورت تصادفی برای مقصد و بقیه به عنوان مبدا است.



شکل ۲-۸: رویکردهای انتخاب توکن‌های مبدا و مقصد در ادغام توکن‌ها

توکن‌های مبدا با رنگ قرمز و توکن‌های مقصد با رنگ آبی مشخص شده‌اند. روش a انتخاب یکی در میان توکن‌ها به عنوان مبدا و مقصد را نشان می‌دهد. روش b انتخاب توکن با گام پرش ۲ را نشان می‌دهد که می‌توانیم به صورت انتخاب توکن بالا سمت راست در یک پنجره‌ی ۲ در ۲ نیز آن را تعریف کنیم. روش c انتخاب ۲۵ درصد از توکن‌ها به صورت تصادفی را نشان می‌دهد و روش d نیز انتخاب تصادفی یک توکن در یک پنجره‌ی ۲ در ۲ را نشان می‌دهد [۲].

بعد از ساخت گراف دوبخشی مطابق شکل ۲-۹ از توکن‌های مبدا و مقصد یک ماتریس مشابهت کسینوسی^{۶۵} بین هر توکن مبدا و توکن‌های مقصد محاسبه می‌شود و بر این اساس r درصد توکن‌ها با بیشترین مشابهت با یکدیگر ادغام می‌شوند. ادغام نیز به صورت میانگین‌گیری میان توکن‌های مورد ادغام انجام می‌شود. فرآیند رفع ادغام^{۶۶} نیز به صورت جایگذاری توکن ادغام شده در تک توکن‌های درگیر در ادغام انجام می‌شود.

شکل ۲-۱۰ محل قرارگیری بلوک‌های ادغام و رفع ادغام را نشان می‌دهد. این بلوک‌ها می‌توانند در اول و آخر بلوک توجه به خود یا توجه متقابل یا بلوک تماماً متصل مبدل‌های موجود در مدل پخشی قرار گیرند. این رویکرد باعث افزایش سرعت صد در صدی در عین حفظ کیفیت می‌شود.

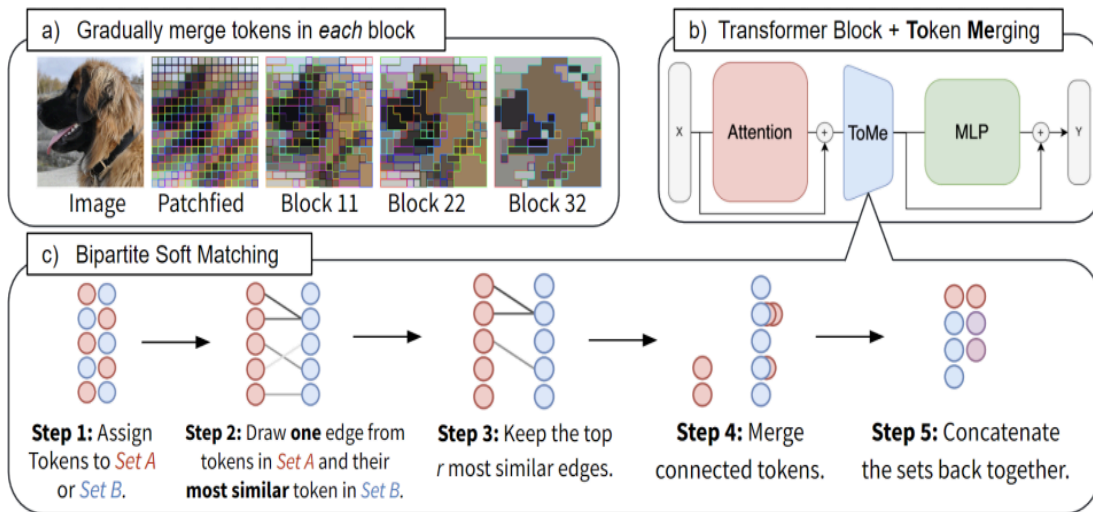
۲-۲-۲-۲-۲ روش همجوشی توکن‌ها

روش همجوشی توکن‌ها^{۶۷} [۹] پس از روش ادغام توکن‌ها و برای مبدل بینایی ارائه شده و سپس روی مدل‌های پخشی نیز تست شد. این روش یک ترکیبی از مزایای ادغام توکن‌ها و هرس آنهاست. روش هرس توکن‌ها معمولاً نیاز به تعریف یک تابع تعیین اهمیت یا امتیازدهی به توکن‌ها دارد و در عین

⁶⁵Cosin Similarity

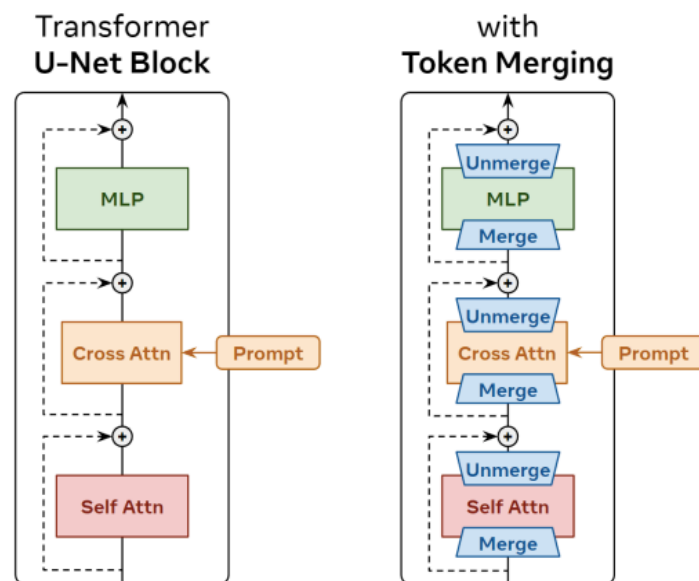
⁶⁶Unmerging

⁶⁷Token Fusion



شکل ۲-۹: روال کار ساخت گراف دوبخشی و ادغام توکن‌ها

مجموعه‌ی A و B در این تصویر همان توکن‌های مبدا و مقصد هستند و این روش مراحل ساخت گراف دوبخشی و انتخاب توکن‌هایی که باهم ادغام می‌شوند را نشان می‌دهد [۱].



شکل ۲-۱۰: محل قرارگیری بلوک‌های ادغام و رفع ادغام در بلوک مبدل در مدل‌های پخشی

همانطور که در شکل مشاهده می‌کنید این بلوک در ابتدا و انتهای بلوک توجه به خود، توجه متقابل یا بلوک تماماً متصل مبدل‌های موجود در مدل پخشی قرار می‌گیرد [۲].

حال حذف بیش از حد در این روش باعث از بین رفتن اطلاعات می‌شود. در مقابل روش ادغام توکن‌ها یک روش بدون نیاز به امتیازدهی است اما در حالتی که ادغام با میانگین‌گیری انجام می‌شود، باعث تغییر

در توزیع داده‌ها می‌شود. از این رو در روش همجوشی توکن‌ها به دنبال یک ایده‌ی بینابینی هستیم. روش همجوشی توکن‌ها بر اساس معیار خطی بودن تابعی^{۶۸} تعریف می‌شود. این معیار بررسی می‌کند که یک مدل تا چه اندازه نسبت به درونیایی ورودی‌هایش خطی عمل می‌کند. در شبکه‌های عصبی یک لایه به تنهایی چندان از معیار خطی بودن تابعی برخوردار نیست اما افزایش عمق و افزودن لایه مخصوصا در مبدل‌های بینایی این معیار را تقویت می‌کند. روش همجوشی از این معیار برای تعیین اینکه در کجا باید از هرس توکن‌ها استفاده شود و در کجا باید از ادغام توکن‌ها استفاده شود، بهره می‌گیرد. در لایه‌هایی که خطی بودن تابعی کم است، استفاده از هرس توکن‌ها بهتر است چرا که ادغام توکن‌ها با میانگین یا هر عملگر دیگری باعث انحراف از توزیع یاد گرفته شده توسط مدل و تغییر در توزیع داده می‌شود؛ و بنابراین استفاده از هرس توکن‌ها که تنها توکن‌های کم اهمیت را حذف می‌کند و درونیایی صورت نمی‌گیرد بهتر است.

در مقابل، در لایه‌هایی که خطی بودن تابعی زیادی دارند، مدل نسبت به ورودی‌های درونیایی شده نتیجه‌ی قابل پیش‌بینی تری دارد و در نتیجه استفاده از روش‌های ادغام توکن‌ها و مزایای آنها در حفظ اطلاعات می‌تواند بهتر باشد. بر این اساس روش همجوشی توکن‌ها با تحلیل میزان برقراری خطی بودن تابعی در لایه‌های اولیه از هرس توکن‌ها و در لایه‌های عمیق‌تر از ادغام توکن‌ها استفاده می‌کند. همچنین با توجه به اینکه میانگین‌گیری باعث کاهش نرم^{۶۹} ویژگی‌ها می‌شود، و این کاهش نرم می‌تواند منجر به تغییر در توزیع شود، این روش به جای میانگین‌گیری در زمان ادغام از معیار ترکیبی درونیایی خطی حداکثر نرم^{۷۰} $MLERP$ استفاده می‌کند که فرمول آن به صورت زیر است:

$$\bar{x} = \frac{1}{K} \sum_{k=1}^K x_k \quad (18-2)$$

$$MLERP(\{(x_1, \dots, x_K)\}) = \frac{\bar{x}}{\|\bar{x}\|} \times \max_{k=1, \dots, K} \|x_k\| \quad (19-2)$$

نتایج پیاده‌سازی این روش بر روی مدل‌های پخشی نشان می‌دهد که این روش نسبت به روش ادغام توکن‌ها کاهش زمانی بیشتری دارد.

⁶⁸Functional Linearity

⁶⁹Norm

⁷⁰Maximum Norm Linear Interpolation

۲-۲-۳ استفاده از حافظه‌ی پنهان

همانطور که پیش‌تر هم اشاره شد روش‌های کاهش گام عمدتاً شبکه‌ی نويززدا را به عنوان یک جعبه‌ی سیاه در نظر می‌گرفتند و بررسی عملکرد بلوک‌های مختلف آن و تاثیر آنها بر سرعت و کیفیت تصویر از جمله مواردی بود که کمتر مورد توجه قرار می‌گرفت. اما ظهور مدل استفاده از حافظه‌ی پنهان عمیق^{۷۱} [۱۶] باعث ایجاد دسته‌ای از روش‌های استفاده از حافظه‌ی پنهان شد که با بررسی افزونگی^{۷۲} موجود در قسمت‌های مختلف مدل‌های پخشی، این رویکرد را در جهت کاهش حجم محاسبات و کاهش زمان نمونه‌برداری اتخاذ کردند.

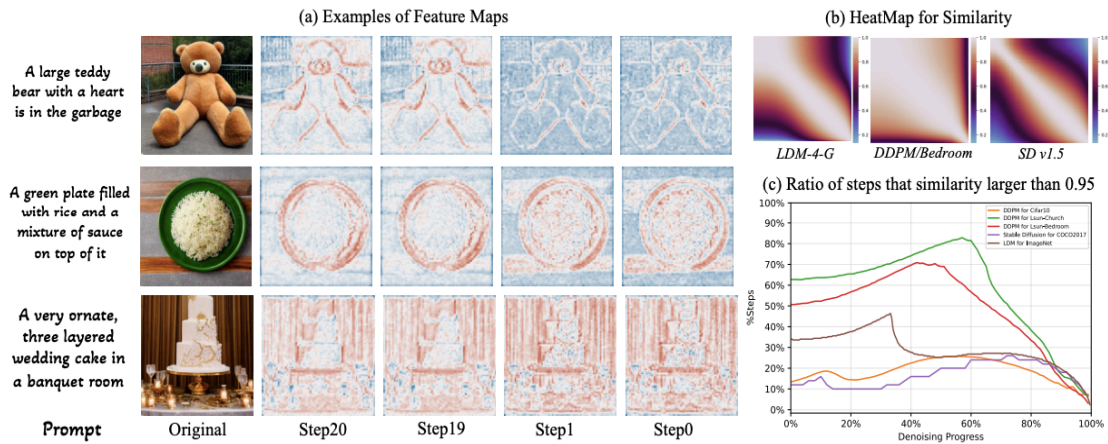
در روش استفاده از حافظه پنهان با اتکا به این موضوع که محاسبات در بلوک‌های مختلف و در گام‌های زمانی مختلف دارای افزونگی هستند، قسمتی از محاسبات را در بلوک مربوطه انجام نمی‌شود و این محاسبات را از بلوک قبلی یا گام زمانی قبلی که در حافظه‌ی پنهان ذخیره شده است جایگذاری می‌شود. روش استفاده از حافظه‌ی پنهان بر ویژگی‌های سطح بالای شبکه‌ی یو، کدگشای شبکه‌ی یو، بلوک توجه متقابل و بر تغییرات محاسبه اعمال شده است که در هر یک از مثال‌های زیر یکی از این حالات را ارائه می‌کنیم.

۲-۲-۳-۱ روش استفاده از حافظه‌ی پنهان عمیق

روش استفاده از حافظه‌ی پنهان عمیق [۱۶] بر مبنای یک مشاهده‌ی تجربی توسعه داده شده است. این مشاهده بیان می‌کند که گام‌های مجاور در فرآیند نويززدایی افزونگی زمانی زیادی را در ویژگی‌های سطح بالا نشان می‌دهند. ویژگی‌های سطح بالا به مجموعه ویژگی‌های حاصل از عبور تصویر از بلوک‌های متوالی کدگذار در شبکه‌ی یو می‌گوییم و منظور از افزونگی زمانی این است که اگر خروجی این بلوک‌ها در دو گام متوالی را مشاهده کنیم تغییرات چشم‌گیری در آنها دیده نمی‌شود. این موضوع را می‌توان به این شکل هم صورت‌بندی کرد که تغییرات در گام‌های متوالی بسیار تدریجی و آهسته است.

شکل ۲-۱۱ این مشاهدات تجربی را نشان می‌دهد. قسمت a خروجی بلوک نمونه‌افزای^{۷۳} U_2 را در دو گام زمانی متوالی نشان می‌دهد که شامل تغییرات خیلی ملموسی نیستند. قسمت b نمودار نقشه حرارتی^{۷۴} مربوط به شباهت را در سه مدل پخشی مختلف نشان می‌دهد و همانطور که مشاهده می‌کنید در نزدیک قطر اصلی یعنی در گام‌های نسبتاً مجاور این عدد شباهت بسیار نزدیک است که موید همان مشاهده‌ی مطرح شده در بند قبل است و در نهایت قسمت c نیز درصد تعداد گام‌هایی که با گام مورد نظر شباهتی بیشتر از ۰.۹۵ دارند را نشان می‌دهد. دقت کنید که تمام نمودارها برای خروجی‌های بلوک نمونه‌افزای U_2 است که در شکل ۲-۱۲ مکان آن را در مدل *Stable Diffusion* می‌توانید مشاهده کنید.

⁷¹Deep Cache⁷²Redundancy⁷³Upsampling⁷⁴Heatmap



شکل ۱۱-۲: مشاهدات تجربی مبنی بر وجود افزونگی زمانی در ویژگی‌های سطح بالا در مدل‌های پخشی. قسمت a نقشه‌ی ویژگی دو مجموعه گام متوالی را در بلوک U_2 نشان می‌دهد. قسمت b نقشه‌ی حرارتی ویژگی‌های مشابهت را در گام‌ها زمانی نشان می‌دهد. قسمت c هم نمودار درصد گام‌هایی که با گام مد نظر مشابهتی بیشتر از ۰.۹۵ را دارند نشان می‌دهد [۱۶].

شکل ۱۲-۲ کلیت این روش را نشان می‌دهد. در این شکل بلوک‌های D_i بلوک‌های زیر نمونه‌گیری^{۷۵} در کدگذار شبکه‌ی یو و بلوک‌های U_i بلوک‌های نمونه افزایش کدگذاری شبکه‌ی یو هستند. این رویکرد بعد از محاسبه‌ی خروجی U_2 برای یک گام $t+1$ و ذخیره‌ی آن در حافظه‌ی پنهان، در گام t به جای محاسبه‌ی ویژگی‌های سطح بالای بلوک‌های میانی آن را از حافظه‌ی پنهان فراخوانی می‌کند و به کمک اتصال پرشی و بلوک U_1 خروجی این گام را محاسبه می‌کند. نکته‌ی دیگری که حائز اهمیت است این است که در چه گام‌هایی محاسبه‌ی مجدد انجام شود و در چه گام‌هایی از حافظه‌ی پنهان فراخوانی شود. در این روش دو رویکرد مطرح می‌شود، رویکرد اول که رویکرد یکنواخت^{۷۶} نام‌گذاری شده است، بیان می‌کند که هر N گام یک بار نیاز است که محاسبات مجدد انجام شوند و مجموعه‌ی $\mathcal{I}$ که گام‌های محاسبه است را به صورت زیر تعریف می‌کند:

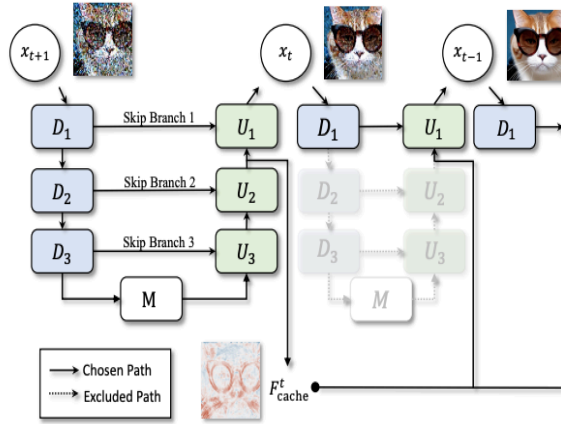
$$\mathcal{I} = \{x \in \mathbb{N} \mid x = iN, 0 \leq i < k\} \quad (20-2)$$

در حالت یکنواخت ما به صورت ضمنی این فرض را داریم که در هر N گام متوالی ویژگی‌های سطح بالا تغییر بسیار ناچیزی دارند، در حالیکه این مخصوصاً در N های بزرگ فرض کاملاً اشتباهی است. با توجه به شکل ۱۱-۲ قسمت c بعد از ۴۰ درصد فرآیند نويز زدایی شباهت زمانی بین ویژگی‌ها کاهش پیدا می‌کند از این رو استفاده از یک رویکرد غیر یکنواخت^{۷۷} برای مشخص کردن مجموعه‌ی $\mathcal{I}$ اهمیت

⁷⁵Downsampling

⁷⁶Uniform

⁷⁷Non-uniform



شکل ۲-۱۲: کلیت روش استفاده از حافظه‌ی پنهان عمیق

شکل کلیت این روش را در دو گام t و $t+1$ نشان می‌دهد. خروجی بلوک U_2 در حافظه‌ی پنهان ذخیره شده و در گام بعدی به جای محاسبه‌ی ویژگی‌های سطح بالا از حافظه‌ی پنهان خروجی گام قبل استفاده شده است [۱۶].

بسیاری دارد. مجموعه‌ی غیر یکنواخت $\mathcal{I}$ به صورت زیر تعریف می‌شود:

$$\mathcal{L} = \left\{ l_i \mid l_i \in \text{linearspace} \left((-c)^{\frac{1}{p}}, (T-c)^{\frac{1}{p}}, k \right) \right\} \quad (21-2)$$

$$\mathcal{I} = \text{uniqueint}(\{i_k \mid i_k = (l_k)^p + c, \text{ where } l_k \in \mathcal{L}\}) \quad (22-2)$$

که c یک پارامتر برای تعیین گام زمانی مرکزی می‌باشد. این روش بر روی $SDv1.5$ با ضریب ۱۱.۲ کاهش سرعت می‌دهد و با اندکی افزایش معیار FID همراه است به این معنا که کیفیت تصویر اندکی کاهش پیدا می‌کند.

۲-۳-۲-۲-۲ مدل پخشی سریع‌تر

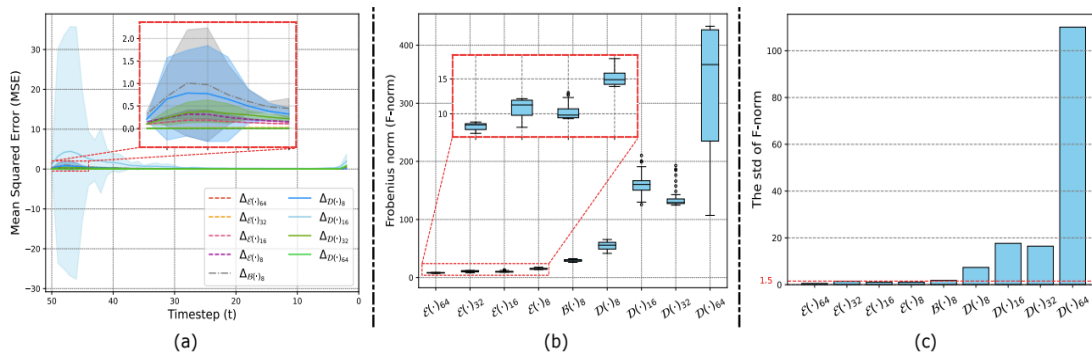
روش مدل پخشی سریع‌تر^{۷۸} [۱۲] سعی دارد تا با ذخیره‌ی خروجی کدگذار شبکه‌ی یوی مدل پخشی، فرآیند نمونه‌برداری مدل‌های پخشی را موازی‌سازی کند. این موضوع مجدداً از این ایده سرچشمه می‌گیرد که به صورت تجربی قابل مشاهده است که ویژگی‌های خروجی کدگذار شبکه‌ی یوی یک مدل پخشی تغییرات بسیار جزئی دارد و در مقابل خروجی کدگشا به طور قابل توجهی تغییر می‌کند. در این روش برای بررسی میزان تغییرات ویژگی‌های شبکه‌ی یو، بلوک‌های این شبکه را به سه دسته‌ی

⁷⁸Faster Diffusion

بلوک‌های کدگذار که شامل $\mathcal{E}(\cdot)_{8}, \mathcal{E}(\cdot)_{16}, \mathcal{E}(\cdot)_{32}, \mathcal{E}(\cdot)_{64}$ و بلوک میانی یا گلوگاه^{۷۹} $\mathcal{B}(\cdot)$ و بلوک‌های کدگشا که شامل $\mathcal{D}(\cdot)_{8}, \mathcal{D}(\cdot)_{16}, \mathcal{D}(\cdot)_{32}, \mathcal{D}(\cdot)_{64}$ است تقسیم می‌کند و سپس با محاسبه‌ی نرم^{۸۰} تفاضل هر بلوک در دو زمان متوالی به فرم زیر معیاری برای شباهت بلوک‌ها در زمان‌های متوالی تعریف می‌کنند.

$$\Delta \mathcal{E}(\cdot)_s = \frac{1}{d \times s^2} \|\mathcal{E}(z_t, c, t)_s - \mathcal{E}(z_{t-1}, c, t-1)_s\|_2^2, \quad (23-2)$$

شکل ۲-۱۳ نمودارهای تجربی به دست آمده از محاسبه‌ی تفاضل بالا در بلوک‌ها و گام‌های زمانی متفاوت را نشان می‌دهد. در قسمت a ترند این تفاضل در گام‌های زمانی متوالی آمده است و نکته‌ی حائز اهمیت مقیاس واریانس تغییر این تفاضل در کدگذار و کدگشا است. در کدگذار واریانس این ترند بین ۰.۴ و ۰.۰۵ تغییر می‌کند و این در حالیست که در کدگشا این عدد بین ۵ تا ۳۰ تغییر می‌کند که نشان دهنده‌ی وجود تغییر بسیار در کدگشا و تغییر بسیار کم در کدگذار است. در قسمت b برای ویژگی‌های خروجی هر بلوک ابتدا میانگین آنها در راستای کانال محاسبه شده و سپس نرم فربینیوس^{۸۱} هر ویژگی محاسبه شده و این عدد برای گام‌های زمانی مختلف محاسبه شده است. همانطور که می‌بینید ارتفاع نمودار جعبه‌ای در کدگذار بسیار ناچیز است و این به این معناست که تغییرات در کدگذار در گام‌های زمانی متفاوت ناچیز است. در قسمت c نیز مشابه قسمت b نرم فربینیوس ویژگی‌ها محاسبه شده است و در این نمودار انحراف معیار آن برای بلوک‌های مختلف مشاهده می‌کنیم. این شکل نیز موید این موضوع است که تغییرات در کدگذار بسیار ناچیزتر از کدگشا هستند.



شکل ۲-۱۳: مشاهدات تجربی مبتنی بر تغییرات ناچیز کدگذار شبکه‌ی یوی مدل پخشی

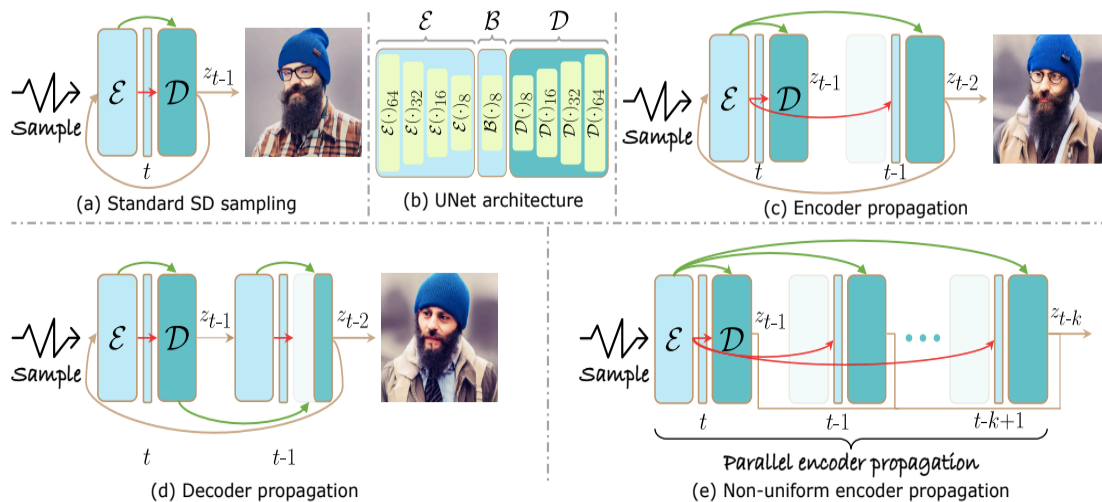
هر سه بخش این تصویر معیارهای برای بازنمایی میزان تغییرات ویژگی‌های خروجی کدگذار در مقابل کدگشا و بلوک میانی هستند. همانطور که مشاهده می‌شود ویژگی‌های کدگذار تغییرات جزئی‌تری نسبت به کدگشا دارند^{۱۲۱}.

⁷⁹Bottleneck

⁸⁰Norm

⁸¹Frobenius Norm

بر مبنای این مشاهدات ایده‌ی اصلی روش مدل پخشی سریع‌تر استفاده از حافظه‌ی پنهان برای نگه‌داری ویژگی‌های کدگذار در بعضی از گام‌ها و محاسبه‌ی آن در سایر گام‌هاست. این ایده که انتشار کدگذار^{۸۲} نام دارد در شکل ۲-۱۴ آمده است. همانطور که در شکل هم قابل مشاهده است، ذخیره‌ی کدگذار در حافظه‌ی پنهان امکان موازی‌سازی فرآیند نویززدایی را هم فراهم می‌آورد. استفاده از این رویکرد برای مدل *Stable Diffusion* با ۴۱ درصد افزایش سرعت در عین حفظ کیفیت همراه است.



شکل ۲-۱۴: کلیت مدل پخشی سریع‌تر

قسمت *a* فرآیند نمونه‌برداری عادی در یک مدل پخشی را نشان می‌دهد. قسمت *b* ساختار یک شبکه‌ی یوی به کار رفته در مدل پخشی را نشان می‌دهد. قسمت *c* فرآیند انتشار کدگذار برای دو گام متوالی را نشان می‌دهد. قسمت *d* فرآیند انتشار کدگذار را نشان می‌دهد که تصویر تولید شده قسمت‌هایی از فرمان داده شده را به خوبی اجرا نمی‌کند. قسمت *e* هم فرآیند موازی‌سازی انتشار کدگذار را نشان می‌دهد [۱۲].

۳-۳-۲-۲-۲ رویکرد گیت T

رویکرد گیت T ^{۸۳} [۱۵] با تمرکز بر بلوک توجه متقابل از حافظه‌ی پنهان استفاده می‌کند. ایده‌ی کلی این روش این است که فرآیند نویززدایی دو قسمت دارد. قسمت اول که فاز برنامه‌ریزی معناییست^{۸۴}، شبکه به فرمان^{۸۵} توجه می‌کند تا با توجه به آن کلیات معنایی تصویر را بسازد و بعد در قسمت دوم که به آن افزایش کیفیت^{۸۶} می‌گوییم، کلیات معنایی را بهبود می‌دهیم و به آنها جزئیات اضافه می‌کنیم. از آنجاییکه مسئولیت روش بلوک توجه متقابل درگیر کردن فرمان در فرآیند تولید نمونه است، ایده‌ی این روش این است که پس از ساخت کلیت معنایی دیگر نیازی به محاسبه‌ی دقیق بلوک توجه متقابل

⁸²Encoder Propagation

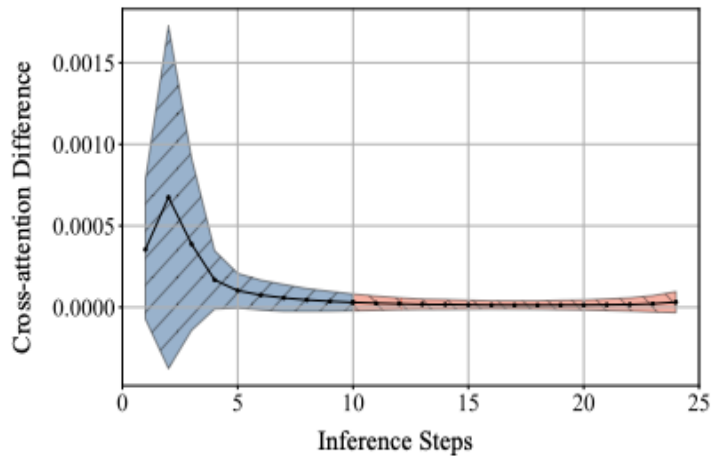
⁸³TGate

⁸⁴Semantic Planning

⁸⁵Prompt

⁸⁶Fidelity Improving

نداریم و می‌توانیم آن را از حافظه‌ی پنهان بازیابی کنیم.

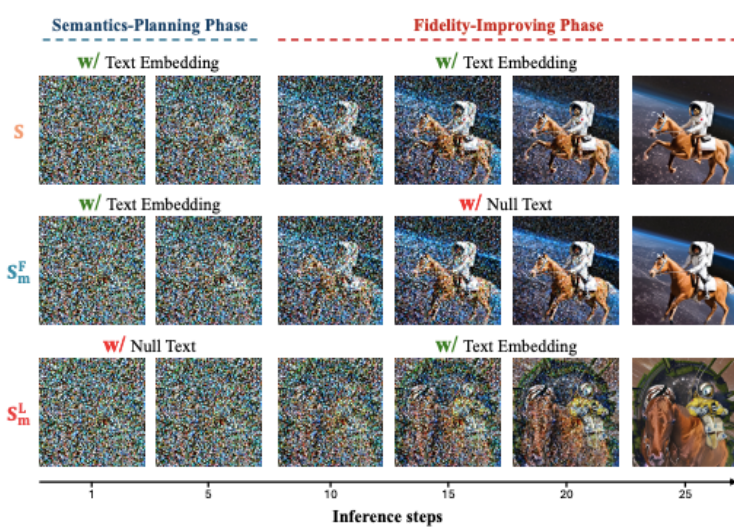


شکل ۲-۱۵: نمودار تفاضل تغییرات بلوک توجه متقابل در گام‌های زمانی متوالی همانطور که نمودار نشان می‌دهد تغییرات در گام‌های ابتدایی بسیار زیاد است و در حول و حوش گام ۵ تا ۱۰ همگرا می‌شود [۱۵].

در این مقاله، دو آزمایش برای بررسی نقش بلوک توجه متقابل انجام شده است. در آزمایش اول مشابه دو روش قبل، تفاضل خروجی بلوک توجه متقابل در گام‌های متوالی برای تعدادی تصویر محاسبه شده است. شکل ۲-۱۵ خروجی این آزمایش را نشان می‌دهد. همانطور که مشاهده می‌کنید، تفاضل خروجی بلوک توجه متقابل در چند گام اول نسبت به گام‌های بعدی بیشتر است و این تفاضل در گام ۵ تا ۱۰ به همگرایی می‌رسد و بعد از آن تقریباً ثابت می‌ماند.

آزمایش دوم برای اثبات نقش بلوک توجه متقابل و نیز برای جداسازی فاز برنامه‌ریزی معنایی از فاز افزایش کیفیت ارائه شده است. این آزمایش به این صورت است که به ازای یک m بین ۰ تا ۲۵ که گام‌های زمانی نویز زدایی هستند مجموعه‌ی گام‌ها را به دو بخش تقسیم می‌کنیم و در یک بخش فرمان را به کمک بلوک توجه متقابل اعمال می‌کنیم و در بخش دوم فرمان را اعمال نمی‌کنیم. سپس تفاوت کیفیت تصویر تولید شده در این حالت با حالت طبیعی مدل پخشی را می‌سنجیم، اگر در چنین حالتی تفاوت ناچیز باشد می‌توانیم نتیجه بگیریم که از گام‌های بعد از m نیازی به اعمال فرمان و محاسبه‌ی توجه متقابل نیست. شکل ۲-۱۶ این آزمایش را نشان می‌دهد.

خروجی این رویکرد به ازای m های مختلف بین ۱۰ تا ۵۰ درصد امکان بهبود سرعت مدل پخشی با حفظ کیفیت را دارد.



شکل ۲-۱۶: تقسیم فرآیند نویززدایی به دو بخش برنامه‌ریزی معنایی و افزایش کیفیت این شکل حالت‌های مختلف حذف فرمان را نشان می‌دهد. همانطور که مشاهده می‌کنید حذف فرمان در گام‌های ابتدایی باعث کاهش چشم‌گیر کیفیت تصویر می‌شود در حالیکه در حالت برعکس کیفیت تصویر تغییر چندانی نمی‌کند [۱۵].

فصل سوم

روش پیشنهادی

در این فصل به معرفی روش پیشنهادی می‌پردازیم. برای ارائه‌ی روش پیشنهاد ابتدا به بررسی دقیق‌تر روش ادغام توکن‌ها که در فصل قبل معرفی شد می‌پردازیم و سپس با بیان نقاط ضعف و قابل بهبود، روش خود را توضیح می‌دهیم. روش پیشنهادی از ترکیب دو ایده‌ی انطباقی ساختن فرآیند ادغام و استفاده از حافظه‌ی پنهان در فرآیند ادغام می‌باشد که در ادامه هر یک را به تفصیل بحث می‌کنیم.

۳-۱ روش پایه

روش پایه‌ی ما در این پژوهش روش ادغام توکن‌ها^۱ [۲] است. این روش ابتدا برای مبدل بینایی و سپس برای مدل‌های پخشی ارائه شد. در این قسمت ابتدا سعی می‌کنیم جزئیات مختلف این روش را بیشتر باز کنیم و به بررسی انتخاب‌های نویسندگان مقاله بپردازیم.

پیش‌تر گفته شد که روش ادغام توکن‌ها بر روی ساختارهای مبدل موجود در مدل پخشی پیاده سازی می‌شود و با توجه به شکل ۲-۱۰ می‌توانیم ببینیم که در هر بلوک مبدل نیز بر سه زیربلوک توجه به خود، توجه متقابل و تماماً متصل قابل اعمال است. اما سوال مهم‌تر این است که به کدامیک از سطوح شبکه‌ی یو قابل اعمال است. شبکه‌ی یو دارای سه لایه‌ی زیرنمونه‌گیری، یک لایه‌ی میانی و سه لایه‌ی نمونه‌افزایی است و در همه‌ی این لایه‌ها بلوک مبدل وجود دارد.

در این روش، لایه‌هایی که مورد فرآیند ادغام قرار می‌گیرند با ابرپارامتری^۲ به نام *maxdownsample* تعیین می‌شود. این پارامتر از تقسیم اندازه‌ی توکن‌های اصلی به توکن‌های ورودی در هر لایه محاسبه می‌شود و می‌تواند مقادیر ۱، ۲، ۴ و ۸ را به خود بگیرد. با انتخاب عدد ۸ ادغام توکن بر روی هر بلوک موجود در شبکه‌ی یو اعمال می‌شود. جدول ۳-۱ نتایج اعمال اعداد مختلف را نشان می‌دهد و بر اساس این مشاهده دو موضوع قابل بیان است اول اینکه افزایش ادغام در لایه‌هایی که ورودی کمتری دارند می‌تواند کیفیت را کاهش دهد و دوم اینکه با توجه به اینکه هر لایه مورد عملیات زیر نمونه‌برداری قرار می‌گیرد و اندازه‌ی آن یک چهارم می‌شود بنابراین سربار اعمال ادغام توکن نسبت به کاهش زمانی که ایجاد می‌کند بیشتر است. و این موضوع سبب می‌شود که فرآیند ادغام توکن‌ها در ویژگی‌های سطح بالا صورت نگیرد.

جدول ۳-۱: نتایج اعمال روش ادغام توکن‌ها بر بلوک‌های مبدل لایه‌های مختلف شبکه‌ی یو
نتایج این جدول نشان می‌دهند که اعمال ادغام توکن بر لایه‌های میانی باعث کاهش کیفیت می‌شود [۲].

تعداد توکن ورودی	بلوک مبدل	معیار $FID \downarrow$	تاخیر (ثانیه) $\downarrow$
۶۴	۱۵ (همه)	۳۵.۶۶	۱.۵۶
۲۵۶	۱۴	۳۵.۷۱	۱.۵۵
۱۰۲۴	۹	۳۴.۳۷	۱.۵۶
۴۰۹۶	۴	۳۳.۲۹	۱.۶۳

¹ToMe

²Hyperparameter

بعد از تعیین لایه‌ها و بلوک‌های مدنظر در مرحله‌ی بعدی برای اعمال ادغام توکن‌ها از فرآیندی به شکلی که پیش‌تر هم بحث شد استفاده می‌شود. در این فرآیند ابتدا توکن‌ها به یک مجموعه‌ی مبدا و مقصد تقسیم می‌شوند تا از روی آنها گرافی دوبخشی ساخته شود. پیش‌تر گفته شد که چهار روش عمده برای تقسیم توکن‌ها به مبدا و مقصد وجود دارد. از بین این روش‌ها انتخاب یک توکن به صورت تصادفی در یک پنجره‌ی ۲ در ۲ بهترین نتیجه‌ی خروجی را می‌دهد. و با توجه به اینکه در هر پنجره‌ی ۲ در ۲ یک توکن برداشته می‌شود تعداد توکن‌های مقصد در بهترین حالت یک چهارم تعداد کل توکن‌های تصویر است و در نتیجه‌ی این موضوع در بیشترین کاهش توکن و با فرض اینکه هر توکن مبدا حتماً به یک توکن مقصد نگاشت می‌شود ما می‌توانیم ۷۵ درصد کاهش داشته باشیم.

بعد از انتخاب توکن‌های مبدا و مقصد در مرحله‌ی بعدی نیاز داریم که ماتریس مشابهت بین توکن‌های مبدا و مقصد را محاسبه کنیم. معیار مشابهت همانطور که پیش‌تر هم اشاره شد، معیار شباهت کسینوسی است. در این مرحله ما به ازای هر مبدل یک بار این ماتریس را محاسبه می‌کنیم که محاسبه‌ی خود این ماتریس نیز برای ما سربار محاسباتی خواهد داشت. و همین سربار محاسباتی مخصوصاً در لایه‌های میانی باعث می‌شود که ادغام توکن‌ها برای ما امری به صرفه نباشد.

پس از محاسبه‌ی مشابهت به ازای هر توکن مبدا، مشابه‌ترین توکن مقصد و میزان مشابهت را پیدا می‌کنیم و r درصد از بیشترین مشابهت‌ها را باهم ادغام می‌کنیم. این انتخاب مستقل از میزان مشابهت‌ها، ساختار عکس ورودی و نواحی غیر متغیر تصویر برای همه‌ی گام‌های زمانی و همه‌ی لایه‌ها یکسان است. در ادامه توضیح خواهیم داد که ثابت در نظر گرفتن r یک نقطه‌ی ضعف و عامل عدم انعطاف رویکرد ادغام توکن‌هاست.

در نهایت نیز توکن‌های انتخاب شده برای ادغام با یکدیگر میانگین گرفته می‌شوند که همانطور که در فصل قبل و در قسمت همجوشی توکن‌ها اشاره شد، میانگین‌گیری می‌تواند باعث تغییر در توزیع و نرم ویژگی‌ها شود و کیفیت را کاهش دهد در همین راستا بهتر است که از عملگرهای دیگری استفاده شود. در جمع‌بندی این قسمت باید گفت که روش ادغام توکن‌ها در کنار بهبود خوبی که ایجاد می‌کند و از روش‌های هرس توکن به تنهایی بهتر است، نقاط ضعفی دارد که بهبود آن‌ها می‌تواند این روش را کاراتر سازد. نقاط ضعف اشاره شده را می‌توان در موردهای زیر خلاصه کرد:

- سربار محاسباتی محاسبه‌ی ماتریس مشابهت می‌تواند باعث شود که این روش بخشی از کاهش زمانی را که به واسطه‌ی ادغام توکن‌ها ایجاد می‌کند، استفاده کند. همچنین در لایه‌های عمیق‌تر شبکه‌ی یو و در ویژگی‌های سطح بالاتر اعمال این روش چندان به صرفه نباشد.
- در نظر گرفتن درصد ادغام r به صورت ثابت برای لایه‌ها و بلوک‌ها و گام‌های زمانی مختلف این روش را انعطاف ناپذیر می‌کند. این درصد ثابت همچنین بدون توجه به ویژگی‌های تصویر و فرکانس‌های مشابهت در نظر گرفته می‌شود.

۲-۳ انطباقی کردن روش ادغام توکن‌ها

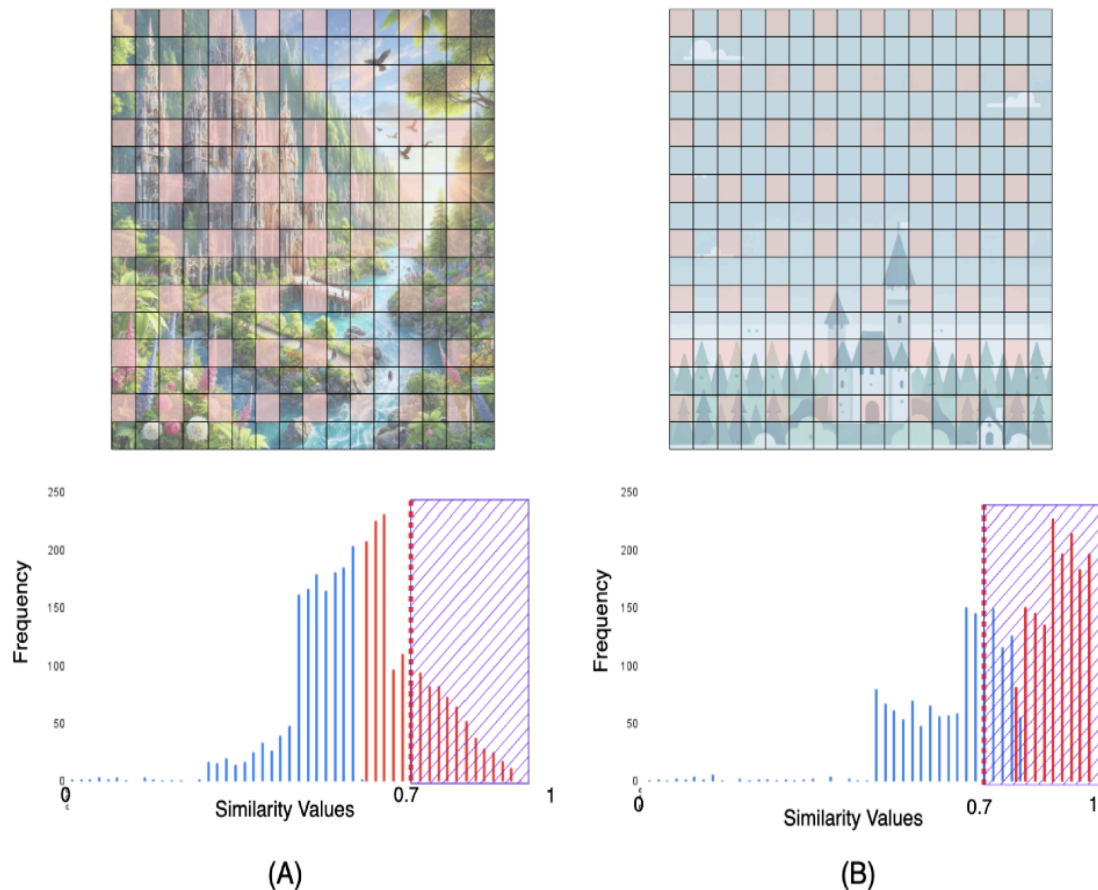
در روش پیشنهادی مطرح شده در این پژوهش و در راستای بهبود روش ادغام توکن‌ها اولین ایده انطباقی^۳ کردن روش پیشنهادی است. با توجه به نقاط ضعفی که در قسمت قبل گفته شد، ثابت بودن درصد ادغام در همه‌ی لایه‌ها و همه‌ی گام‌های زمانی و بدون در نظر گرفتن تصویر ورودی باعث عدم انعطاف روش ادغام توکن‌ها می‌شود.

شکل ۱-۳ این موضوع را به صورت نمادین نشان می‌دهد. در نمودار a هیستوگرام مشابهت توکن‌های مبدا و مقصد یک تصویر در فضای پنهان با توکن‌های متنوع را داریم. این تصویر جزئیات زیادی دارد و تغییرات متعددی در پیکسل‌های متوالی آن وجود دارد. قسمت قرمز نمودار ۵۰ درصد توکن‌ها را که با یکدیگر ادغام خواهند شد نشان می‌دهد. همانطور که می‌بینیم در این مثال در روش ادغام توکن‌ها بدون در نظر گرفتن توزیع آنها حجم زیادی از توکن‌هایی که مشابهت خیلی کمتری هم دارند را ادغام می‌کنیم که این امر می‌تواند باعث از دست رفتن اطلاعات در تعدادی از توکن‌های مبدا که به اشتباه با توکن مقصدی که شباهت زیادی با آن ندارند ادغام شده‌اند می‌شود.

در حالت دیگر که در قسمت b تصویر ۱-۳ آمده است، نمودار هیستوگرام یک تصویر با جزئیات کم و توکن‌های نسبتاً مشابهی را داریم که تغییرات توکن‌ها نیز در این تصویر کم است. در این حالت با توجه به خاصیت تصویر می‌توانیم حجم بیشتری توکن را بدون از دست رفتن اطلاعات ادغام کنیم در حالیکه ۵۰ درصد توکن‌ها که با قسمت قرمز نمودار مشخص شده‌اند خیلی کمتر از امکان ادغام واقعی هستند. بنابراین در نظر گرفتن یک درصد ثابت برای ادغام بدون اهمیت به توزیع کلی مشابهت‌های موجود می‌تواند باعث عدم کارایی روش ادغام توکن‌ها گردد.

راه‌حل تغییر این عدم انعطاف موجود در روش ادغام توکن‌ها تعریف یک حد آستانه برای انتخاب توکن‌هایی که با هم ادغام می‌شوند می‌باشد. در این راه‌حل با تعریف میزانی که برای مشابهت عدد مناسبی است می‌توانیم هر تعداد توکنی که از حد آستانه مشابه‌ترند را با یکدیگر ترکیب کنیم. خط‌چین‌های قرمز در قسمت a و b تصویر ۱-۳ آستانه‌ی ۰.۷ را برای ادغام توکن‌ها نشان می‌دهند. در این حالت معایبی که پیش‌تر و به واسطه‌ی تعریف درصد ثابت ادغام با آنها مواجه بودیم را برطرف می‌کند. در روش پیشنهادی ما از تعریف یک آستانه‌ی t برای ادغام به جای استفاده از درصد ادغام در روش ادغام توکن‌ها استفاده شده است.

³Adaptive



شکل ۳-۱: تصویر نمادین تفاوت حالت انطباق پذیر و حالت ثابت ادغام توکن‌ها
این تصویر دو حالت که در آن استفاده از یک آستانه برای ادغام نسبت به در نظر گرفتن درصد ثابت مزیت بیشتری دارد را نشان می‌دهد.

۳-۳ استفاده از حافظه‌ی نهان در ادغام توکن‌ها

موضوع دیگری که باعث عدم کارایی روش ادغام توکن‌ها می‌شود محاسبه‌ی ماتریس مشابهت توکن‌هاست. این ماتریس با توجه به اینکه برای هر مبدا و در هر گام زمانی محاسبه می‌شود می‌تواند سربار محاسباتی زیادی به روش ادغام توکن‌ها تحمیل کند.

از طرفی با توجه به ایده‌ی اصلی که در عمده‌ی روش‌های مبتنی بر حافظه‌ی پنهان داریم که به افزودن زمانی و تغییرات بسیار آهسته و هموار در خروجی لایه‌های مختلف اشاره می‌کند می‌توانیم از این موضوع برای کم کردن سربار محاسباتی روش ادغام توکن‌ها استفاده کنیم.

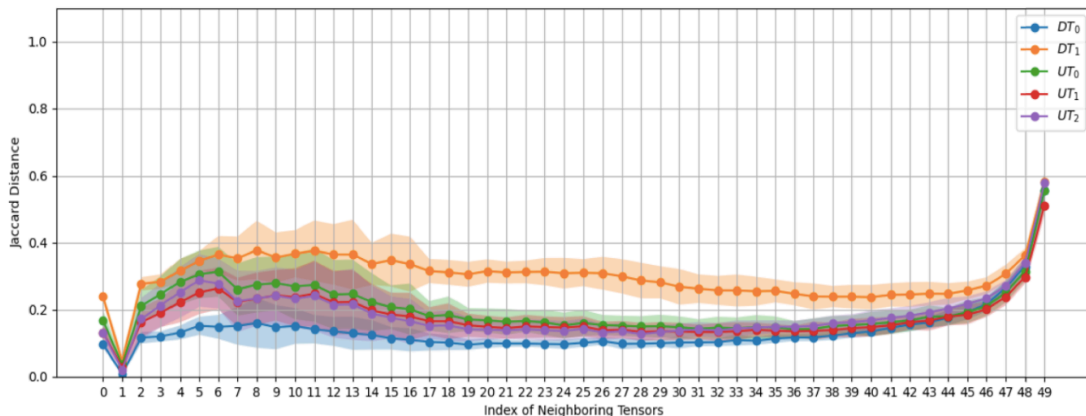
در روش‌های معرفی شده برای رویکرد استفاده از حافظه‌ی پنهان دیدیم که در اکثر قسمت‌های شبکه‌ی یوی نویزدا تغییرات به صورت ناچیز و هموار رخ می‌دهند و اساساً همین مشاهده باعث ایجاد این دسته از روش‌هاست. ایده‌ی اصلی ما نیز در این بخش بر همین مبناست که اگر میزان تغییرات در خروجی بلوک‌های مختلف شبکه‌ی یو در گام‌های زمانی مختلف بسیار کم است پس الگویی که برای

ادغام توکن‌های خروجی هر یک از این بلوک‌ها که در واقع توکن‌های ورودی بلوک بعدی هستند استفاده می‌شود نیز تغییر چندانی نخواهد داشت و بنابراین می‌توانیم به جای محاسبه‌ی ماتریس مشابهت و به دنبال آن جفت‌های توکن‌های مبدا و مقصد، آنها را از حافظه‌ی پنهان به دست آوریم و فقط در برخی از گام‌ها آنها را محاسبه کنیم.

برای بررسی درستی فرضیه‌ی اولیه‌ای که در خصوص ناچیز بودن تغییرات روش ادغام در بلوک‌های متناظر داریم، از معیار فاصله‌ی جاکارد^۴ مجموعه‌ی جفت‌های ادغام شده در دو گام متوالی استفاده می‌کنیم. معیار فاصله‌ی جاکارد برای دو مجموعه‌ی A_n که جفت توکن‌های مشابه در گام n هستند و A_{n+1} که جفت توکن‌های مشابه در گام $n+1$ هستند، تعریف می‌شود و میزان مشابهت اعضای آنها را به صورت زیر محاسبه می‌کند:

$$JaccardDistance(A_n, A_{n+1}) = 1 - \frac{|A_n \cap A_{n+1}|}{|A_n \cup A_{n+1}|} \quad (۱-۳)$$

این معیار برای گام‌های زمانی صد عکس از صد کلاس مختلف *ImageNet* و در پنج بلوک مبدل با $MaxDownsample = 1$ در شکل ۲-۳ رسم شده است. همانطور که مشاهده می‌کنید میزان تغییرات در اکثر گام‌ها چندان چشم‌گیر نیست و این موضوع فرضیه‌ی ما مبنی بر هموار بودن تغییرات ماتریس مشابهت و جفت توکن‌های ادغامی را تایید می‌کند.



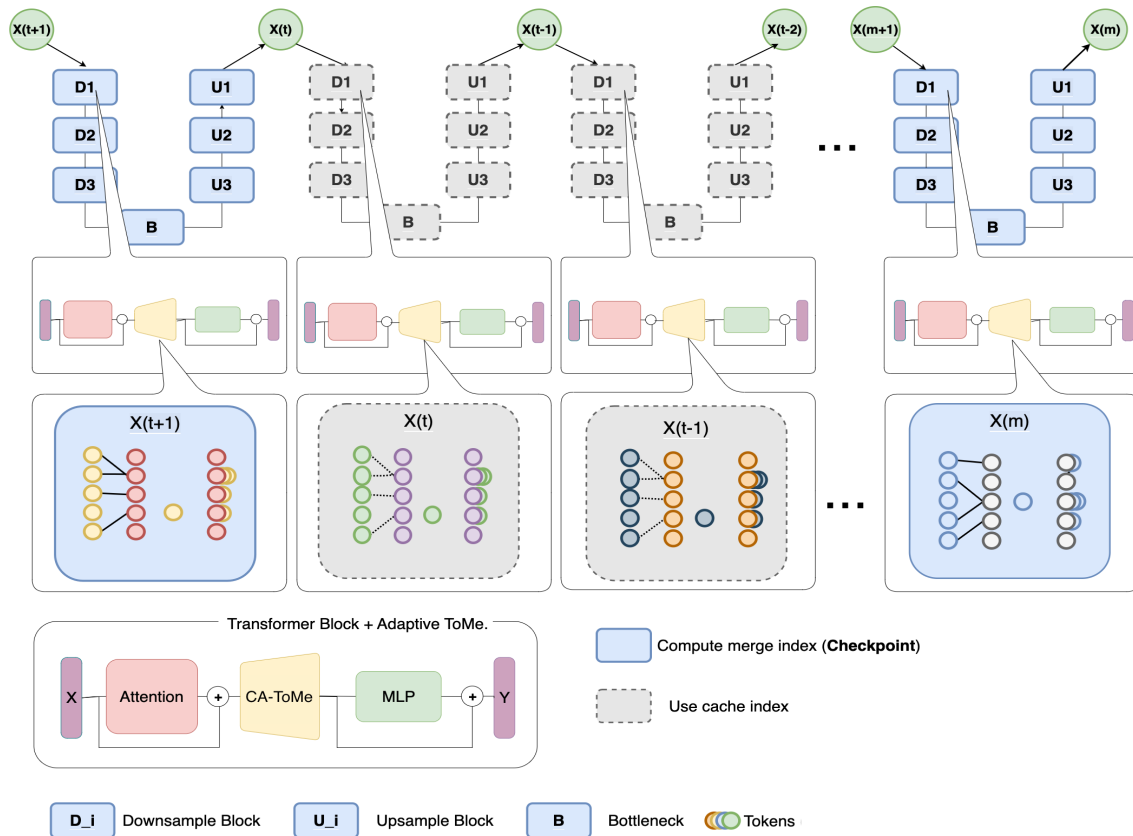
شکل ۲-۳: نمودار فاصله‌ی جاکارد مجموعه‌ی جفت‌های ادغام در روش ادغام توکن‌ها

این نمودار فاصله‌ی جاکارد جفت‌های ادغام هر دو گام متوالی برای صد عکس تولید شده از صد کلاس *ImageNet* را نشان می‌دهد. همچنین نواحی هاشور خورده اطراف هر نمودار واریانس مقادیر را نشان می‌دهد.

بر اساس این مشاهده می‌توانیم فرآیند کامل ادغام را در تعدادی از گام‌ها انجام دهیم و در سایر گام‌ها از نمایه‌های جفت‌های ادغام شده در مراحل قبلی استفاده کنیم و بدون محاسبه‌ی مجدد ماتریس مشابهت آنها را با یکدیگر ادغام کنیم. شکل ۳-۳ این روش را به صورت شماتیک نشان می‌دهد. در این

^۴Jaccard Distance

شکل گام‌هایی که با رنگ آبی مشخص شده‌اند بیانگر گام‌هایی هستند که محاسبات در آنها به صورت کامل انجام می‌شود و گام‌هایی که با رنگ خاکستری مشخص شده‌اند گام‌هایی را مشخص می‌کنند که در آنها جفت‌های قابل ادغام از حافظه‌ی پنهان خوانده می‌شود.



شکل ۳-۳: شمای کلی روش پیشنهادی و استفاده از حافظه‌ی پنهان در فرآیند ادغام توکن‌ها

در این شکل بلوک روش پیشنهادی ما با $CA - ToMe$ نشان داده شده است که مخفف همان روش ادغام توکن تطبیقی مبتنی بر حافظه‌ی پنهان^۵ است. همچنین گام‌های آبی نمایانگر گام‌هایی هستند که در آنها محاسبات انجام می‌شود و گام‌های خاکستری نمایانگر گام‌هایی هستند که در آنها محاسبه از حافظه‌ی پنهان خوانده می‌شود.

در نهایت، روش پیشنهادی که روش ادغام توکن تطبیقی مبتنی بر حافظه‌ی پنهان نام دارد، در راستای برطرف کردن نقاط ضعف روش ادغام توکن ارائه شده است. این روش به طور کلی دو نوآوری اصلی دارد:

- تطبیقی کردن فرآیند ادغام با تعریف یک آستانه برای میزان مشابهت توکن‌هایی که برای ادغام برگزیده می‌شوند.
- استفاده از حافظه‌ی پنهان بر مبنای این ایده که تغییرات توکن‌ها در گام‌های متوالی ناچیز است و ذخیره‌ی ماتریس مشابهت و نمایه‌های جفت‌های قابل ادغام در حافظه‌ی پنهان.

فصل چهارم

آزمایش‌ها و نتایج

در این فصل ابتدا تنظیمات آزمایش‌ها و روش‌های ارزیابی را معرفی می‌کنیم و سپس به ارائه‌ی نتایج و تحلیل و بررسی آنها می‌پردازیم.

۱-۴ مجموعه داده

به طور کلی و در مقالات مطالعه شده، هر نوع داده‌ی تصویری حجیم دارای برچسب برای ارزیابی امر تولید تصویر به کار گرفته می‌شود اما دو دسته داده را در اکثر پژوهش‌های اخیر در بحث استفاده از حافظه‌ی پنهان و کاهش توکن به کار گرفته شده‌اند که در ادامه هر یک را معرفی می‌کنیم و فرآیند ارزیابی با هر یک را ذکر می‌کنیم.

۱-۱-۴ مجموعه داده‌ی *ImageNet*

مجموعه داده‌ی *ImageNet* [۳] یک مجموعه داده‌ی تصویری بزرگ مقیاس و به صورت عمومی در دسترس است که بر اساس ساختار شبکه‌ی کلمات^۱ توسعه داده شده است. در شبکه‌ی کلمات هر مفهوم معنادار با تعدادی کلمه‌ی هم‌معنی تعریف می‌شود که به آن مجموعه‌ی هم‌معنی^۲ گفته می‌شود و در حدود ۸۰ هزار مجموعه‌ی هم‌معنی موجود در شبکه‌ی کلمات اسم هستند. تلاش سازندگان مجموعه داده‌ی *ImageNet* بر این است که به ازای هر مجموعه‌ی هم‌معنی موجود حدود ۱۰۰۰ تصویر جمع‌آوری کنند.

پرکاربردترین زیرمجموعه از *ImageNet* که در کاربردهای مختلفی از دسته‌بندی گرفته تا تولید تصاویر به کار می‌رود، مجموعه‌ایست شامل ۱۰۰۰ کلاس داده‌ی مختلف و شامل یک میلیون و دویست هزار داده‌ی آموزشی ۵۰ هزار داده‌ی صحت سنجی و ۱۰۰ هزار داده‌ی آزمون. این مجموعه داده در سایت *Kaggle* در دسترس است و مجموعه داده‌ایست که در ارزیابی ما نیز مد نظر بوده است.

۲-۱-۴ مجموعه داده‌ی *COCO*

مجموعه داده‌ی *COCO* [۱۴] یا اشیای رایج در پس زمینه‌ی ماکروسافت^۳ یک مجموعه داده‌ی بزرگ مقیاس دیگر است که برای انواعی از مسائل بینایی ماشین از قطعه‌بندی معنایی، شناسایی اشیای تا مسئله‌ی تولید تصاویر به کار گرفته می‌شود. این مجموعه شامل برچسب اشیای، مستطیل دربرگیرنده‌ی آنها و متن^۴ توصیف کننده‌ی اشیای است و در مجموع شامل بیش از ۳۳۰ هزار تصویر و بیش از ۸۰ شی مختلف می‌باشد.

^۱ WordNet

^۲ Synonym Set

^۳ Microsoft Common Objects in Context

^۴ Caption

در قسمت ارزیابی علت انتخاب مجموعه داده‌ی *ImageNet* را توضیح خواهیم داد اما علاوه بر این دو مجموعه داده، مجموعه داده‌های دیگری مانند *CIFAR10* [۱۱]، *LSUN - bedroom* و *LSUN - Church* نیز وجود دارند که به علت کم بودن تعداد اشیا و کلاس‌های موجود در آنها و خاص منظوره بودن آنها در این ارزیابی لحاظ نشده‌اند.

۲-۴ معیارهای ارزیابی

مسالهی مطرح در این پژوهش دو معیار اساسی و مهم دارد که این دو معیار در یک مصالحه هستند. به طور کلی هدف ما افزایش سرعت است و همانطور که در قسمت‌های پیشین گفته شد این افزایش سرعت از طریق کاهش حجم محاسبات شبکه‌ی نوپزدا به دست می‌آید. اما نکته‌ی مهم این است که این کاهش حجم محاسبات نباید کیفیت تصاویر تولیدی را تحت‌الشعاع قرار دهد. از این رو دو معیار اساسی در این پژوهش زمان و کیفیت تصاویر تولیدی هستند.

برای اندازه‌گیری زمان همان زمان میانگین تولید تصاویر در فرآیند نمونه برداری در نظر گرفته شده‌است. عیب اصلی این معیار این است که بر حسب نوع سخت‌افزار مورد استفاده ممکن است تفاوت کند. در این آزمایش تمامی نتایج بر روی پردازشگر گرافیکی *V100* انجام شده است و زمان‌های گزارش شده اعدادی هستند که در این تنظیمات سخت افزاری مشاهده شده‌اند. همچنین میزان کاهش زمان نسبت به مدل پایه نیز به صورت ضریب گزارش شده است تا حالت نرمال شده را نیز داشته باشیم.

برای تعیین کیفیت از معیار فاصله‌ی اکتسابی فرچت^۵ یا *FID* [۶] استفاده می‌کنیم. این معیار از شباهت آماری بین تصاویر اصلی و تصاویر تولید شده برای ارزیابی کیفیت تصاویر تولید شده استفاده می‌کند. این معیار با به دست آوردن بازنمایی ویژگی‌های تصاویر دو مجموعه با استفاده از یک شبکه‌ی از پیش آموزش دیده‌ی *inception* و محاسبه‌ی فاصله‌ی فرچت روی این بازنمایی‌های به دست آمده، معیار مد نظر را محاسبه می‌کند. فاصله‌ی فرچت، بر اساس میانگین و کوواریانس مقایسه می‌کند که توزیع ویژگی‌های تصاویر تولید شده تا چه اندازه به تصاویر اصلی نزدیک است. رابطه‌ی ریاضیاتی آن به صورت زیر می‌باشد:

$$FID = \|\mu_r - \mu_g\|^2 + Tr(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}), \quad (1-4)$$

که در این فرمول μ_r و μ_g به ترتیب میانگین بازنمایی ویژگی‌های تصاویر اصلی و تصاویر تولید شده هستند. همچنین Σ_r و Σ_g هم ماتریس کوواریانس این ویژگی‌ها هستند. هر چقدر مقدار *FID* کمتر باشد نمایان‌گر کیفیت بهتر تصاویر تولیدی و شباهت بیشتر آن با مجموعه‌ی تصاویر واقعی می‌باشد. این معیار همبستگی خوبی با درک بینایی انسان از کیفیت دارد.

با وجود این که در اکثر مقالات مرتبط با تولید تصاویر معیار *FID* گزارش می‌شود؛ این معیار محدودیت‌های

⁵Frechet Inception Distance

زیادی دارد که به برخی از آنها در ادامه اشاره می‌کنیم:

- این معیار یک معیار آماریست و با توجه به اینکه از تخمین میانگین و کوواریانس استفاده می‌کند، نیاز داریم که حجم داده‌ی ورودی و نمونه‌های زیادی داشته باشیم تا محاسبات قابل اتکا باشد.
- با وجود اینکه معیار FID از نظر کلیات مشابهت دو مجموعه داده با درک بینایی انسان همبستگی دارد در جزئیات و مصنوعات اضافه شده به تصور می‌تواند تفاوت زیادی داشته باشد.
- شرایط اندازه‌گیری و پیش‌پردازش‌های انجام شده در محاسبه‌ی این معیار در مقدار آن موثر است به این معنا که به ازای اندازه‌ی نمونه‌های مختلف و پیش‌پردازش‌های متفاوتی از تصاویر خروجی از یک شبکه می‌توانیم معیار FID متفاوتی بگیریم.

برای رفع محدودیت‌های معیار FID دو رویکرد به کار گرفته می‌شود. اول، ثابت نگه داشتن شرایط آزمایش برای مدل‌های مبتنی بر یک مدل پایه و دوم، استفاده از سایر معیارها مانند $Precision$, $LPIPS$ و $Recall$ که در ادامه به کارگیری هر دوی این رویکردها در بهبود آزمایشات این پژوهش را توضیح می‌دهیم.

برای یکسان‌سازی شرایط آزمایش با سایر آزمایش‌های افزایش سرعت مدل‌های پخشی بسته به نوع داده دو روش وجود دارد. در محاسبه‌ی FID بر روی داده‌ی $COCO$ نیاز به تولید ۳۰ هزار عکس و مقایسه‌ی آنها با مجموعه‌ی صحت‌سنجی مجموعه داده‌ی $COCO$ داریم که با توجه به اینکه تولید هر هزار عکس در حدود دو ساعت زمان می‌برد استفاده از این تنظیمات آزمایشی از نظر زمانی امکان پذیر نبود. برای محاسبه‌ی FID در مجموعه داده‌ی $ImageNet$ یک روش استاندارد وجود دارد که در مقالات کاهش توکن از این روش استفاده می‌شود. در این روش از هر کلاس مجموعه داده‌ی $ImageNet$ دو تصویر با فرمان " $A High Quality Image of (ClassLabel)$ " تولید می‌کنیم و سپس برای مجموعه داده‌ی اصلی نیز ۵ تصویر از هر کلاس را از مجموعه داده‌ی آموزش می‌خوانیم که در مجموع ۵۰۰۰ عکس می‌شود. در این شرایط که آزمایش ما هم به همین صورت انجام شده است، نتایج قابلیت مقایسه پذیری با روش‌های کاهش توکن دارند.

برای افزایش دقت محاسبات یک معیار دیگر را نیز در تصمیم‌گیری در خصوص انتخاب بهترین تنظیمات و نیز در بحث مقایسه‌ی مدل‌ها در نظر گرفتیم. این معیار مشابهت ادراکی یادگرفته شده‌ی قطعات تصویر^۶ یا $LPIPS$ [۲۶] نام دارد و برای سنجش کیفیت با معیاری نزدیکتر به درک بینایی انسان به کار می‌رود.

^۶Learned Perceptual Image Patch Similarity

۳-۴ تعیین ابرپارامترها

برای بررسی نتایج از مدل *Stable Diffusion* ورژن ۵.۱ استفاده شده است و تغییرات بر روی تابع *forward* شبکه‌ی یوی این مدل اعمال شده است. همانطور که در بخش روش پیشنهادی توضیح داده شد، روش ما نیاز به تعیین دو ابرپارامتر دارد. اول مقدار آستانه‌ی t برای انطباقی کردن فرآیند ادغام توکن‌ها و نیز پارامتر *config* که تعیین کننده‌ی نمایه‌ی گام‌هایست که در آنها فرآیند ادغام توکن‌ها مجدد محاسبه می‌شود.

برای تعیین آستانه‌ی t با توجه به اینکه مقدار مشابهت کسینوسی بین صفر و یک است و با توجه به این موضوع که به ازای هر توکن مبدا توکن مقصدی که بیشترین مشابهت را با آن دارد انتخاب می‌شود و فاصله‌ی این دو در نظر گرفته می‌شود، آستانه معمولاً از مقدار ۰.۴ بیشتر و از یک کمتر است. بنابراین در این پژوهش با استفاده از روش تجربی و محاسبه‌ی معیارهای کیفیت و زمان برای آستانه‌های مختلف مقدار این ابرپارامتر تعیین شده است.

جدول ۴-۱ نتایج آزمایشات برای حالت بدون اعمال استفاده از حافظه‌ی پنهان و آستانه‌های مختلف را نشان می‌دهد. انتظار داریم که با افزایش آستانه که به این معناست که اجازه‌ی ادغام را برای توکن‌های بسیار مشابه می‌دهیم و شرط سخت‌گیرانه‌تری برای ادغام داریم، زمان هم افزایش پیدا کند چراکه توکن‌های کمتری ادغام می‌شوند و ورودی مبدل کاهش کمتری دارد. از طرفی در آستانه‌ها پایین‌تر اجازه‌ی ادغام تمامی توکن‌ها را می‌دهیم که باعث کاهش زمان و کیفیت می‌شود. بنابراین باید آستانه‌ای انتخاب شود که مصالحه‌ای میان کیفیت و زمان باشد. همانطور که مشاهده می‌شود مقدار معیار *FID* برای آستانه‌ی ۰.۷ با توجه به کاهش زمان این آستانه مناسب و نزدیک به روش ادغام توکن‌هاست.

جدول ۴-۱: معیارهای کیفیت و زمان برای آستانه‌های مختلف در روش ادغام تطبیقی

آستانه	<i>FID</i>	میانگین زمان (ثانیه)
۰.۴	۳۵.۲۸	۶.۰۷ ± ۰.۰۰۷
۰.۵	۳۵.۴۶	۶.۰۷ ± ۰.۰۰۴
۰.۶	۳۵.۵۶	۶.۱۰ ± ۰.۰۰۵
۰.۷	۳۴.۳۰	۶.۲۳ ± ۰.۰۰۲
۰.۸	۳۳.۸۰	۶.۵۸ ± ۰.۰۰۴
۰.۹	۳۳.۴۲	۶.۹۲ ± ۰.۰۰۳
۱.۰	۳۳.۶۶	۷.۶۱ ± ۰.۰۰۱
ادغام توکن‌ها	۳۴.۱۶	۶.۳۹ ± ۰.۰۰۶

برای تعیین گام‌های محاسبه‌ی ادغام توکن‌ها در مقابل گام‌هایی که این محاسبات از حافظه‌ی پنهان بارگذاری می‌شوند، با توجه به کارهای انجام شده در سایر آزمایشات و کارهای پیشین و نیز با توجه به تصویر ۲-۳ که فاصله‌ی جاکارد را در گام‌های زمانی مختلف نشان می‌دهد، ۶ تنظیم را در نظر گرفته‌ایم که سعی می‌کنند حالت‌های مختلف تغییرات را نشان دهند. این تنظیمات در جدول ۴-۲ آمده است. به عنوان مثال *CONFIG*₁ با فرض اینکه تغییرات در گام‌های ابتدایی بیشتر است و به مرور زمان جزئی‌تر

می‌شود، در گام‌های ابتدایی محاسبات بیشتری لحاظ می‌کند و در گام‌های انتهایی بیشتر از حافظه‌ی پنهان استفاده می‌کند.

جدول ۴-۲: گام‌های محاسبه در تنظیمات مختلف در نظر گرفته شده در روش ادغام توکن تطبیقی مبتنی بر حافظه‌ی پنهان

نام تنظیمات	گام‌های محاسبه
$CONFIG_1$	[0, 1, 2, 3, 5, 10, 15, 25, 35]
$CONFIG_2$	[0, 10, 11, 12, 15, 20, 25, 30, 35, 45]
$CONFIG_3$	[0, 8, 11, 13, 20, 25, 30, 35, 45, 46, 47, 48, 49]
$CONFIG_4$	[0, 9, 13, 14, 15, 28, 29, 32, 36, 45]
$CONF_{ADAPTIVE}$	[0, 1, 5, 7, 10, 12, 15, 35, 40, 45, 46 – 51]
$CONF_{FIVE}$	[0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50]

برای هر یک از تنظیمات در نظر گرفته شده و با تعیین مقدار آستانه با عدد ۰.۷ که به عنوان آستانه‌ی بهینه انتخاب شده است، آزمایشات تولید ۲۰۰۰ تصویر از کلاس‌های مجموعه داده‌ی *ImageNet* انجام شده است و نتایج این آزمایشات در جدول ۴-۳ آمده‌است. همانطور که مشاهده می‌شود، تنظیم $CONF_3$ بهترین خروجی را هم از نظر کیفی و هم از نظر زمانی تولید می‌کند.

جدول ۴-۳: عملکرد کلی روش ادغام توکن مبتنی بر حافظه‌ی پنهان در تنظیمات گام‌های محاسبه‌ی متفاوت

روش	آستانه/درصد	تنظیمات حافظه پنهان	میانگین زمان (ثانیه)	FID
روش پیشنهادی	۰.۷	$CONF_1$	6.18 ± 0.02	۳۵.۱۴
روش پیشنهادی	۰.۷	$CONF_2$	6.13 ± 0.001	۳۳.۸۴
روش پیشنهادی	۰.۷	$CONF_3$	6.09 ± 0.001	۳۳.۴۹
روش پیشنهادی	۰.۷	$CONF_4$	6.12 ± 0.001	۳۴.۱۸
روش پیشنهادی	۰.۷	$CONF_{ADAPTIVE}$	6.19 ± 0.001	۳۴.۷۷
روش پیشنهادی	۰.۷	$CONF_{FIVE}$	6.14 ± 0.001	۳۴.۵۷
مدل پخشی پایه	-	بدون حافظه پنهان	7.61 ± 0.001	۳۳.۳۷
ادغام توکن‌ها	۰.۵	بدون حافظه پنهان	6.39 ± 0.006	۳۳.۴۸

۴-۴ تحلیل و بررسی نتایج

نتایج نهایی جمع‌بندی شده با بهترین تنظیمات هر مدل در جدول ۴-۴ آمده است. این جدول نشان می‌دهد که روش ادغام توکن مبتنی بر حافظه‌ی پنهان با حفظ کیفیت می‌تواند در مدل پایه با ضریب ۱.۲۴ افزایش سرعت ایجاد کند و این افزایش نسبت به روش ادغام توکن‌ها نیز بیشتر است.

همچنین با توجه به جدول ۴-۱ در حالتی که تمام توکن‌های مبدا و مقصد را با یکدیگر ادغام کنیم، اتفاقی که در آستانه‌ی ۰.۴ می‌افتد، زمان حداقل به ۶.۰۷ ثانیه می‌رسد و در ازای این ادغام کامل دو

جدول ۴-۴: مقایسه‌ی مدل پایه و مدل ادغام توکن‌ها با روش پیشنهادی در بهترین تنظیمات آن

مدل / روش	<i>FID</i>	زمان میانگین (ثانیه)
مدل <i>Stable Diffusion v1.5</i>	۳۳.۳۷	7.61 ± 0.001
روش ادغام توکن‌ها	۳۳.۴۸	6.39 ± 0.006
روش پیشنهادی	۳۳.۴۹	6.09 ± 0.001

واحد افزایش در معیار *FID* داریم که به معنای کاهش کیفیت است. اما روش ما بدون ادغام تمامی توکن‌ها با این مقدار زمانی تنها ۰.۰۲ ثانیه فاصله دارد در حالیکه کیفیت آن به هیچ عنوان افت نمی‌کند. نکته‌ی قابل نتیجه‌گیری از این موضوع این است که اگر بدون در نظر گرفتن کیفیت بخواهیم حداکثر توان این روش را برای کاهش زمان نمونه‌برداری محاسبه کنیم این عدد از حالتی به دست می‌آید که تمامی توکن‌ها را ادغام کرده باشیم یعنی در نظر گرفتن آستانه‌ی ساده‌گیرانه‌ی ۰.۴ و بنابراین روش پیشنهادی ما سعی می‌کند به این حداکثر ظرفیت کاهش زمان با این روش برسد در حالیکه کیفیت تصاویر تولیدی را ثابت نگه میدارد.

نکته‌ی دیگری که حائز اهمیت است این است که این روش بر اکثر روش‌های بهبود سرعت دیگر عمود است و می‌تواند در کنار روشی مانند استفاده از حافظه‌ی پنهان عمیق قرار گیرد تا به افزایش سرعت بیشتری برسیم. همچنین این روش می‌تواند در ترکیب با سایر روش‌های کاهش گام نیز بهبود بیشتری ایجاد کند.

کتاب نامه

- [1] Bolya, Daniel, Fu, Cheng-Yang, Dai, Xiaoliang, Zhang, Peizhao, Feichtenhofer, Christoph, and Hoffman, Judy. Token merging: Your vit but faster. arXiv preprint arXiv:2210.09461, 2022.
- [2] Bolya, Daniel and Hoffman, Judy. Token merging for fast stable diffusion. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pages 4599–4603, 2023.
- [3] Deng, Jia, Dong, Wei, Socher, Richard, Li, Li-Jia, Li, Kai, and Fei-Fei, Li. Imagenet: A large-scale hierarchical image database. In 2009 IEEE Conference on Computer Vision and Pattern Recognition, pages 248–255, 2009.
- [4] Feller, William. Retracted chapter: On the theory of stochastic processes, with particular reference to applications. In Selected Papers I, pages 769–798. Springer, 2015.
- [5] Haurum, Joakim Bruslund, Escalera, Sergio, Taylor, Graham W, and Moeslund, Thomas B. Which tokens to use? investigating token reduction in vision transformers. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pages 773–783, 2023.
- [6] Heusel, Martin, Ramsauer, Hubert, Unterthiner, Thomas, Nessler, Bernhard, and Hochreiter, Sepp. Gans trained by a two time-scale update rule converge to a local nash equilibrium. Advances in neural information processing systems, 30, 2017.

-
- [7] Hinton, Geoffrey. Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531, 2015.
- [8] Ho, Jonathan, Jain, Ajay, and Abbeel, Pieter. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- [9] Kim, Minchul, Gao, Shangqian, Hsu, Yen-Chang, Shen, Yilin, and Jin, Hongxia. Token fusion: Bridging the gap between token pruning and token merging. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 1383–1392, 2024.
- [10] Kreis, Karsten, Gao, Ruiqi, and Vahdat, Arash. Denoising diffusion-based generative modeling: Foundations and applications, 2022.
- [11] Krizhevsky, Alex, Hinton, Geoffrey, et al. Learning multiple layers of features from tiny images. 2009.
- [12] Li, Senmao, Hu, Taihang, Shahbaz Khan, Fahad, Li, Linxuan, Yang, Shiqi, Wang, Yaxing, Cheng, Ming-Ming, and Yang, Jian. Faster diffusion: Rethinking the role of unet encoder in diffusion models. arXiv e-prints, pages arXiv–2312, 2023.
- [13] Li, Xiuyu, Liu, Yijiang, Lian, Long, Yang, Huanrui, Dong, Zhen, Kang, Daniel, Zhang, Shanghang, and Keutzer, Kurt. Q-diffusion: Quantizing diffusion models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 17535–17545, 2023.
- [14] Lin, Tsung-Yi, Maire, Michael, Belongie, Serge, Hays, James, Perona, Pietro, Ramanan, Deva, Dollár, Piotr, and Zitnick, C Lawrence. Microsoft coco: Common objects in context. In *Computer Vision—ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part V 13*, pages 740–755. Springer, 2014.

- [15] Liu, Haozhe, Zhang, Wentian, Xie, Jinheng, Faccio, Francesco, Xu, Mengmeng, Xiang, Tao, Shou, Mike Zheng, Perez-Rua, Juan-Manuel, and Schmidhuber, Jürgen. Faster diffusion via temporal attention decomposition. arXiv e-prints, pages arXiv–2404, 2024.
- [16] Ma, Xinyin, Fang, Gongfan, and Wang, Xinchao. Deepcache: Accelerating diffusion models for free. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 15762–15772, 2024.
- [17] Ma, Zhiyuan, Zhang, Yuzhu, Jia, Guoli, Zhao, Liangliang, Ma, Yichao, Ma, Mingjie, Liu, Gaofeng, Zhang, Kaiyan, Li, Jianjun, and Zhou, Bowen. Efficient diffusion models: A comprehensive survey from principles to practices. arXiv preprint arXiv:2410.11795, 2024.
- [18] Peebles, William and Xie, Saining. Scalable diffusion models with transformers. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pages 4195–4205, 2023.
- [19] Rombach, Robin, Blattmann, Andreas, Lorenz, Dominik, Esser, Patrick, and Ommer, Björn. High-resolution image synthesis with latent diffusion models. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pages 10684–10695, 2022.
- [20] Ronneberger, Olaf, Fischer, Philipp, and Brox, Thomas. U-net: Convolutional networks for biomedical image segmentation. In Medical image computing and computer-assisted intervention–MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18, pages 234–241. Springer, 2015.
- [21] Salimans, Tim, Karpathy, Andrej, Chen, Xi, and Kingma, Diederik P. Pixelcnn++: Improving the pixelcnn with discretized logistic mixture likelihood and other modifications. arXiv preprint arXiv:1701.05517, 2017.

- [22] Shang, Yuzhang, Yuan, Zhihang, Xie, Bin, Wu, Bingzhe, and Yan, Yan. Post-training quantization on diffusion models. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pages 1972–1981, 2023.
- [23] Song, Jiaming, Meng, Chenlin, and Ermon, Stefano. Denoising diffusion implicit models. arXiv preprint arXiv:2010.02502, 2020.
- [24] Song, Yang and Ermon, Stefano. Generative modeling by estimating gradients of the data distribution. Advances in neural information processing systems, 32, 2019.
- [25] Xiao, Zhisheng, Kreis, Karsten, and Vahdat, Arash. Tackling the generative learning trilemma with denoising diffusion gans. arXiv preprint arXiv:2112.07804, 2021.
- [26] Zhang, Richard, Isola, Phillip, Efros, Alexei A, Shechtman, Eli, and Wang, Oliver. The unreasonable effectiveness of deep features as a perceptual metric. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 586–595, 2018.

واژه‌نامه‌ی فارسی به انگلیسی

Prior distribution توزیع اولیه	ا
Generator تولید کننده	اتصالات پرشی Skip connections
ج	Variance exploding انفجار واریانس
Normalizing flows جریان‌های نرمال‌ساز	ب
ح	Residual block بلوک باقی‌مانده‌ای
Evidence lower bound حد پایین مشاهده	ت
Variational bound حد تغییراتی	تابع امتیاز Score function
Euler-maruyama solver حل‌کننده‌ی اوایلر ماریاما	تابکاری شبیه‌سازی شده Simulated annealing
خ	Mixing ترکیب
Variational autoencoders خودکدگذارهای تغییراتی	ترمیم تصویر Image inpainting
خودکدگذارهای تغییراتی سلسله مراتبی	تعبیه Embedding
Hierarchical variational autoencoders	تعیین یکتا Unique identifiability
د	تطبیق امتیاز Score matching
Likelihood درست‌نمایی	تطبیق امتیاز نویزدا Denoising score matching
Langevin dynamics دینامیک لانژوین	تمایزگر Discriminator

Semantic segmentation قطعه‌بندی معنایی	Annealed Langevin dynamics دینامیک لانژوین تابکاری شده
ک	KL-divergence دیورجنس KL
Decoder کدگشا	ر
Encoder کدگذار	Colorization رنگ‌آمیزی
ل	ش
Log-likelihood لگاریتم درست‌نمایی	Generative adversarial networks شبکه‌های مولد تقابلی
م	
Mode مد	U-net شبکه‌ی یو
مدل پخشی نویززدای احتمالاتی	ض
Denoising diffusion probabilistic models	
Score-based generative models مدل‌های مولد مبتنی بر امتیاز	Diffusion coefficient ضریب پخش
Latent variable models مدل‌های متغیر پنهان	Drift coefficient ضریب رانش
غ	
Diffusion models مدل پخشی	Intractable غیرقابل محاسبه
Forward trajectory مسیر روبه جلو	ف
مشکل سه‌گانه‌ی یادگیری مولد	Frechet inception distance فاصله‌ی اکتسابی فرچت
Generative learning trilemma	
Ordinary differential equations معادلات دیفرانسیل ساده	Deterministic process فرآیند قطعی
Posterior mean میانگین پسین	Forward process فرآیند رو به جلو
Vector field میدان برداری	Standard wiener process فرآیند وینر استاندارد
ن	
Anomaly ناهنجاری	Support فضای پشتیبان
Diffusion rate نرخ پخش	ق

Saddle point نقطه‌ی زینی

Importance نمونه‌برداری اهمیت
sampling

ه

Kernel هسته

Binomial kernel . . . هسته‌ی دو جمله‌ای

Transition kernel هسته‌ی گذر

واژه‌نامه‌ی انگلیسی به فارسی

A	Discrimination تمایزگر
Anomaly ناهنجاری	Drift coefficient ضریب رانش
Annealed دینامیک لانژوین تابکاری شده	E
langevin dynamics	Encoder کدگذار
B	Embedding تعبیه
Binomial kernel هسته‌ی دوجمله‌ای	Evidence lower حد پایین مشاهده
C	bound
Colorization رنگ‌آمیزی	Euler- حل‌کننده‌ی اوایلر ماریاما
D	maruyama solver
Decoder کدگشا	F
Denoising score تطبیق امتیاز نویززا	Forward process فرآیند رو به جلو
matching	Forward trajectory مسیر روبه جلو
مدل پخشی نویززدای احتمالاتی	Frechet فاصله‌ی اکتسابی فرچت
Denoising diffusion probabilistic models	inception distance
Deterministic process فرآیند قطعی	G
Diffusion coefficient ضریب پخش	Generative شبکه‌های مولد تقابلی
Diffusion models مدل پخشی	adversarial networks
Diffusion rate نرخ پخش	مشکل سه‌گانه‌ی یادگیری مولد
	Generative learning trilemma

Generator تولید کننده	Ordinary معادلات دیفرانسیل ساده
H	differential equations
خودکدگذارهای تغییراتی سلسله مراتبی	P
Hierarchical variational autoencoders	Posterior mean میانگین پسین
I	Prior distribution توزیع اولیه
Image inpainting ترمیم تصویر	R
Importance نمونه‌برداری اهمیت	Residual block بلوک باقی‌مانده‌ای
sampling	S
Intractable غیرقابل محاسبه	Saddle point نقطه‌ی زینی
K	Score- مدل‌های مولد مبتنی بر امتیاز
Kernel هسته	based generative models
KL-divergence KL دیورجنس	Score function تابع امتیاز
L	Score matching تطبیق امتیاز
Langevin dynamics دینامیک لانژوین	Semantic قطعه‌بندی معنایی
Latent variable مدل‌های متغیر پنهان	segmentation
models	Semi-supervised یادگیری نیمه نظارتی
Likelihood درست‌نمایی	learning
Log-likelihood لگاریتم درست‌نمایی	Simulated تابکاری شبیه‌سازی شده
M	annealing
Mixing ترکیب	Skip connections اتصالات پرشی
Mode مد	Standard wiener فرآیند وینر استاندارد
N	process
Normalizing flows جریان‌های نرمال‌ساز	Support فضای پشتیبان
O	T

Transition kernel هسته‌ی گذر	Variance exploding انفجار واریانس
U	Variational . . . خودکدگذارهای تغییراتی
U-net شبکه‌ی یو	autoencoders
Unique identifiability . . . تعیین یکتا	Variational bound حد تغییراتی
V	Vector field میدان برداری



Amirkabir University of Technology
(Tehran Polytechnic)

Department of ...

M. Sc. Thesis

Title of Thesis

By

Name Surname

Supervisor

Dr.

Advisor

Dr.

Month & Year